Pega CSSA Interview Bank

Senior System Architect · Constellation UI · Pega Infinity

308 highly technical, real-world scenario questions · Scenario / Question / Ideal answer focus

This bank is written from the perspective of a Lead Pega Architect and senior technical interviewer. Every item follows a three-part structure: a **Scenario** (a realistic performance, migration, integration, security, or UX problem), the targeted **Question**, and the **Ideal answer focus** — the specific guardrails, rules, and Constellation mechanisms a strong candidate must reference to pass. Terminology reflects the current Pega Infinity / Constellation platform (DX API v2, view-based authoring, stateless client orchestration, queue processors and job schedulers rather than agents, PAL/PDC/DB Trace, and rule resolution).

The four pillars below map to the competency areas requested. Use the ‘ideal answer focus’ as a scoring rubric: candidates who hit the named mechanisms demonstrate genuine architectural depth rather than surface familiarity.

Pillars

Pillar A — Case Design & Life Cycle Management

Q1

Scenario: A retail bank's account-opening case has grown to 9 stages, 40+ processes, and dozens of decision shapes embedded directly in flows. Every small change requires regression across the whole case, and business users can no longer read the lifecycle.

Question: **How would you refactor this monolithic case type, and what design principles keep the lifecycle maintainable?**

> **Ideal answer focus:** Center-out / Microjourney thinking: split into focused case types with child cases for independently-progressing work; keep stages as business milestones only; move branching logic out of flows into decision tables/when rules; use reusable processes/subflows; favor App Studio case designer readability; avoid deep flow nesting. Guardrail: minimize customization, maximize reuse.

Q2

Scenario: A case must move to a 'Fraud Hold' path from ANY point in the lifecycle the moment a fraud signal is received, then return to where it was.

Question: **How do you implement a jump-from-anywhere behavior without wiring connectors from every shape?**

> **Ideal answer focus:** Tickets: place a ticket at the fraud-handling stage/point and raise it programmatically (or via a Declare rule / listener) so the flow jumps regardless of current position; discuss alternate stages for the hold; note that resuming to prior position needs a stored return point. Mention Change Stage vs ticket trade-offs.

Q3

Scenario: Cases occasionally get stuck: an assignment sits forever because the flow reached a state with no valid connector, and users can't proceed.

Question: **How do you diagnose a 'flow with no valid next step' problem and prevent it by design?**

> **Ideal answer focus:** Diagnose with Tracer (flow execution, failed connectors) and the case's flow status; ensure every decision/connector set has an 'otherwise' path; validate likely-when conditions; use 'no matching connector' handling; add a broken-process flow / FlowProblems handling. Guardrail: always define default outgoing paths.

Q4

Scenario: An approval must escalate through three management levels based on the requested amount, and the number of levels changes by region.

Question: **How do you implement configurable multi-level approvals that business can adjust without rule changes?**

> **Ideal answer focus:** Cascading approvals driven by a decision table / authority matrix returning approver levels; use reporting-structure routing (manager of); externalize thresholds to application settings; avoid hard-coded operator IDs; keep it declarative so business edits the table.

Q5

Scenario: A step should only execute for 'Premium' customers; for others the whole stage is irrelevant and should be bypassed automatically.

Question: **How do you conditionally skip a process or an entire stage, and where does the condition live?**

> **Ideal answer focus:** Skip-a-stage / skip-a-process with a when condition (start-when / skip-when); reusable when rule; ensure the case still transitions correctly after skipping; note that stage automatic transition continues. Avoid empty assignments as pseudo-skips.

Q6

Scenario: Users need to perform ad-hoc actions (update contact info, add a note, cancel) at any time during a long-running case, independent of the current assignment.

Question: How do you expose these without disrupting the primary flow?

> **Ideal answer focus:** Case-wide optional actions (local actions) available from the case, not tied to a connector; distinguish local vs connector flow actions; secure with privileges; ensure they don't advance the main flow; in Constellation these surface as case actions.

Q7

Scenario: A parent 'Order' case spins off child 'Shipment' cases. The parent must not resolve until all shipments complete, but must also react immediately if any shipment is cancelled.

Question: How do you coordinate parent/child dependencies for both 'wait for all' and 'react to one'?

> **Ideal answer focus:** Case dependencies / Wait step (wait for all children resolved) plus a ticket or Declare OnChange/Trigger to react to a cancelled child; discuss child status propagation; avoid circular dependencies; consider dependency configuration vs custom logic.

Q8

Scenario: Duplicate service requests from the same customer are flooding the queue, but the current duplicate check flags too many false positives.

Question: How do you tune duplicate case detection for precision?

> **Ideal answer focus:** Search Duplicate Cases with weighted + must-match conditions; tune weights and threshold; choose the right basic/weighted criteria (customer + item + time window); resolve action (merge/relate/reject). Guardrail: configure, don't custom-code matching.

Q9

Scenario: A case must pause until an external partner confirms via callback, which may take hours or days, and the requestor session cannot stay open.

Question: How do you model a long asynchronous wait correctly?

> **Ideal answer focus:** Wait step (case dependency / timer) not an open assignment; the case persists and resumes on the external event (service callback updates the case / raises a ticket); avoid holding locks/sessions; note Constellation statelessness means no reliance on live session.

Q10

Scenario: Business wants the case status shown to end users to be meaningful ('Awaiting documents', 'In underwriting') and drive reporting.

Question: How do you design case statuses across the lifecycle?

> **Ideal answer focus:** Set pyStatusWork meaningfully per stage/step; use Resolved-* to close; ensure statuses are reportable (Insights); keep a controlled status vocabulary; tie to stage entry. Guardrail: use standard status conventions.

Q11

Scenario: After go-live, SLAs seem inconsistent: some cases escalate on weekends, others don't, and different teams expect different working hours.

Question: How do you make SLA timing respect business calendars and vary by team?

> **Ideal answer focus:** Service Level with business calendar (skip weekends/holidays); calendar per org unit/operator; goal vs deadline vs passed-deadline intervals; circumstance or parameterize the SLA per team; verify calendar assignment on operators/workbaskets.

Q12
Scenario: A high-priority case must always be picked before others, but Get Next Work keeps handing agents lower-priority items.

Question: **How does urgency and Get-Next-Work prioritization actually work, and how do you fix it?**

> **Ideal answer focus:** Urgency = assignment urgency (SLA-driven + manual); Get Next Work uses urgency and workbasket/merge criteria; check GetNextWork settings, urgency calculation, and whether items are on worklist vs workbasket; adjust SLA initial urgency and escalation. Avoid overriding standard GNW logic unless necessary.

Q13
Scenario: An assignment SLA must send two reminders, then reassign to a backup queue, then keep escalating urgency until done.

Question: **How do you stage a multi-interval escalation on one SLA?**

> **Ideal answer focus:** One Service Level with goal/deadline + multiple passed-deadline intervals; earlier intervals = notify (correspondence), later = reassign (to workbasket) + urgency bump; repeating passed-deadline interval to keep escalating. Reference escalation activities.

Q14
Scenario: The organization reorganizes frequently; routing that named specific operators now breaks constantly.

Question: **How do you design routing resilient to org changes?**

> **Ideal answer focus:** Route to work queues and by reporting relationship / operator skills, not operator IDs; use org hierarchy (ToWorkbasket, reporting-manager routing); skill-based routing; keep routing decisions in when/decision rules. Guardrail: avoid hard-coded users.

Q15
Scenario: A regulated process requires that the person who submits a request can never be the person who approves it, even if they hold the approver role.

Question: **How do you enforce segregation of duties in routing?**

> **Ideal answer focus:** Routing logic excludes the submitter (compare pxCreateOperator / requestor); route to manager or alternate approver; combine with privilege checks; document as a business rule in a when/decision rule; audit via history.

Q16
Scenario: Business wants to run two independent review processes in a stage simultaneously and only continue when both finish.

Question: **How do you model parallel processing with a join?**

> **Ideal answer focus:** Parallel processes/steps in the stage (or a split-join / spinoff of subflows) with a join condition; ensure both complete before stage transition; discuss parallel vs sequence; watch locking when both touch the same case.

Q17
Scenario: A screen-flow wizard collects data across 5 screens for one user, but users need to jump back and edit earlier screens without losing later data.

Question: **How do you build multi-step navigation while preserving entered data?**

> **Ideal answer focus:** Screen flow / multi-step form with navigation enabling back/forward; data persists on the case page across steps; validate per step vs on submit; in Constellation, multi-step processes render as a stepper. Avoid losing clipboard data between steps.

Q18
Scenario: A case type is reused by three divisions, but each needs a slightly different approval step and correspondence.

Question: **How do you specialize behavior per division without cloning the case type?**

> **Ideal answer focus:** Circumstancing (by division property) on the specific rules (flow action, correspondence, SLA); class-based specialization via ECS layers; keep base case type shared; relevant records for reuse. Guardrail: specialize the rule, not the whole case.

Q19
Scenario: A resolved case must be reopenable under audit, restarting only the relevant stage, with full history retained.

Question: **How do you implement controlled case reopening?**

> **Ideal answer focus:** Reopen action moving case back to an appropriate stage/status; preserve audit history (pxHistory); guard with privilege; set status back from Resolved-*; ensure declarative/SLA reinitialization as needed.

Q20
Scenario: Stakeholders keep changing stage order during workshops and want to see the impact immediately.

Question: **How do you validate lifecycle changes rapidly and safely?**

> **Ideal answer focus:** App Studio case designer visual lifecycle (or Pega Blueprint) for live iteration; DCO/Pega Express; branch rulesets for parallel dev; unit tests on flows; avoid editing locked prod rulesets directly.

Q21
Scenario: A flow calls a subprocess as a spin-off, but downstream logic assumes the spun-off work is already complete, causing intermittent data issues.

Question: **What is the difference between spin-off and call, and how do you fix this race condition?**

> **Ideal answer focus:** Spin-off runs in parallel (does not wait); call waits for completion. Fix: use a call/subprocess if the result is needed before continuing, or add a Wait/dependency on the spun-off child. Discuss synchronization and case locking.

Q22
Scenario: An automated step calls a connector that occasionally times out, leaving the case in an inconsistent partially-processed state.

Question: **How do you make an integration-bearing step resilient and idempotent?**

> **Ideal answer focus:** Wrap in error handling (flow error transition / FlowProblems), route to exception path; retry with backoff (queue processor for async); design idempotency (avoid duplicate side effects on retry); decouple via data page; set connector timeouts. Guardrail: never let a case silently stall.

Q23
Scenario: A CSA built a large activity with 30 steps to orchestrate a case; it's now unmaintainable and scores poorly on guardrails.

Question: **How would you re-architect procedural logic into best-practice constructs?**

> **Ideal answer focus:** Replace with case lifecycle steps, data transforms, decision tables, declarative rules, and automations; reserve activities for genuinely unsupported logic; raise compliance score; improve testability. Guardrail: minimize activities / custom code.

Q24
Scenario: Users complain a case 'jumps' unpredictably between stages in production but not in dev.

Question: **How do you troubleshoot non-deterministic stage transitions?**

> **Ideal answer focus:** Check for raised tickets, change-stage actions, automatic vs manual transitions, and circumstanced rules resolving differently per environment/user; use Tracer + case history; verify ruleset versions promoted; check when-rule inputs differ by data.

Q25
Scenario: A case needs to create 500 child cases from a bulk upload without freezing the user's browser.

Question: **How do you create many child cases efficiently and asynchronously?**

> **Ideal answer focus:** Queue the creation via a queue processor / bulk (async) processing rather than synchronously in the flow; batch creates; pass data to each child; monitor via queue dashboards; avoid synchronous loops that block the request/DX call.

Q26
Scenario: Business wants an SLA on the overall case AND on individual assignments, and they sometimes conflict.

Question: **How do case-level and assignment-level SLAs interact, and how do you design them coherently?**

> **Ideal answer focus:** Case/stage SLA governs total resolution; assignment SLA governs each task; both can run concurrently; ensure escalations don't contradict; set case SLA looser than sum of assignment goals; document precedence.

Q27
Scenario: A flow action submits successfully in the UI but the case doesn't advance; no error is shown.

Question: **How do you diagnose a 'submit succeeds but no progression' issue?**

> **Ideal answer focus:** Check connector conditions/likely-when, post-processing validation silently failing, flow action pre/post activities, and (Constellation) whether the DX API response indicates validation messages; Tracer the flow action and connector evaluation.

Q28
Scenario: An optional action must be available only to supervisors and only while the case is in a specific stage.

Question: **How do you constrain optional action availability by role and stage?**

> **Ideal answer focus:** Local action with a privilege (supervisor) + availability condition (stage/when rule); in Constellation these are case actions filtered by security and visibility; avoid exposing then denying at submit.

Q29
Scenario: A process must wait for a fixed SLA-driven timer but also allow a manual 'proceed now' override by a manager.

Question: **How do you combine a timed wait with a manual override?**

> **Ideal answer focus:** Wait step (timer) plus an optional/local action or ticket that lets a manager resume immediately; ensure the timer is cancelled on override; secure the override with a privilege.

Q30
Scenario: The business wants a single 'complaint' case type to handle wildly different complaint categories with different steps.

Question: **How do you model divergent paths within one case type cleanly?**

> **Ideal answer focus:** Use decision-driven stage/process selection, alternate stages, or child case types per category; avoid one flow with excessive branching; consider separate case types if divergence is large. Balance reuse vs clarity.

Q31
Scenario: After migrating from a section/harness app, the team wants to keep complex flow logic but modernize the experience.

Question: **What case-design changes are required to be Constellation-ready?**

> **Ideal answer focus:** Ensure DX API compliance: view-based UI, data model-driven; remove reliance on harness/section-only features, custom HTML, and non-auto-generated UI; flows/case logic largely reusable but UI must be Views; screen layout as harness container. Guardrail: DX API compliance.

Q32
Scenario: A case has a step that must run different automations depending on channel (web vs mobile vs email).

Question: **How do you vary step behavior by channel in a Constellation world?**

> **Ideal answer focus:** Circumstance rules by the x-origin-channel header value; channel-aware configuration; keep business logic channel-agnostic (center-out) and only presentation/behavior varies. Avoid encoding channel logic redundantly.

Q33
Scenario: Business requires a full audit trail of who did what and when on every case for compliance.

Question: **How do you ensure comprehensive, tamper-evident case auditing?**

> **Ideal answer focus:** Leverage standard case history (pxHistory), field-level auditing (track security changes / audit note), and status changes; avoid custom logging; use history for reporting; consider security event logging. Guardrail: use built-in history.

Q34
Scenario: A long case accumulates a huge clipboard page with many embedded lists, hurting performance on each save.

Question: **How do you keep case data lean over a long lifecycle?**

> **Ideal answer focus:** Model reference data via data pages instead of embedding copies; avoid unbounded embedded page lists on the case; use child cases for large sub-datasets; expose only needed properties; watch BLOB size.

Q35
Scenario: A step should resolve the case automatically only if all validations pass, otherwise route to manual review.

Question: **How do you branch on validation outcome within the lifecycle?**

> **Ideal answer focus:** Validate rule / Declare Constraint producing messages; decision on validation result routes to auto-resolve vs manual review assignment; keep validation server-side; avoid client-only checks.

Q36
Scenario: Two agents open the same case simultaneously; one's changes silently overwrite the other's.

Question: **How does locking work in Constellation, and how do you handle concurrent edits?**

> **Ideal answer focus:** Constellation uses optimistic locking via eTag/if-match (pxSaveDateTime); a stale update is rejected with a conflict; design UI to handle 409/refresh; contrast with traditional pessimistic locking; choose locking strategy on the case type.

Q37
Scenario: The client wants certain cases to auto-purge/archive after resolution per data-retention policy.

Question: **How do you implement retention/archival for resolved cases?**

> **Ideal answer focus:** Use Pega's archival/purge (retention policies, purge/archive job schedulers) rather than custom deletes; ensure history/audit retention rules; performance and compliance considerations.

Q38
Scenario: A case type must be delivered as an MLP in 6 weeks, then iterated. Stakeholders keep adding scope.

Question: **How do you scope and sequence delivery as an architect?**

> **Ideal answer focus:** Pega Express / DCO: identify the microjourney MLP, defer non-essential stages, iterate; use Blueprint for fast design; manage scope via user stories in Agile Workbench; deliver end-to-end thin slice first.

Q39
Scenario: A subprocess is reused across case types but one case type needs a slightly different post-step.

Question: **How do you reuse the subprocess while allowing localized variation?**

> **Ideal answer focus:** Keep the shared subprocess; add the variation via circumstancing, an extension point, or a case-specific wrapper step; avoid copying the whole flow. Guardrail: reuse with specialization.

Q40
Scenario: Business wants to A/B two different routing strategies and measure resolution time.

Question: **How do you implement and evaluate alternative routing designs?**

> **Ideal answer focus:** Circumstance/route by a strategy flag; capture timing via SLA/case data; report with Insights; iterate; keep both strategies configurable. Data-driven decision.

Q41
Scenario: A case reopened after resolution retriggers SLAs and correspondence that should not fire again.

Question: **How do you prevent unwanted re-firing of declarative/SLA logic on reopen?**

> **Ideal answer focus:** Guard SLA start and correspondence with when conditions (e.g., not previously sent / reopen flag); reset only intended timers; use flags on the case; test reopen path explicitly.

Q42
Scenario: A case type must support both a 'fast track' (auto-approve) and a 'full review' path decided at intake, and business changes the eligibility rules monthly.

Question: **How do you design switchable straight-through vs manual paths that business can maintain?**

> **Ideal answer focus:** Decision (decision table/when) at intake selecting the path; straight-through process (no assignments) vs review stage; externalize eligibility to a business-editable decision table; keep both paths as configured processes. Guardrail: declarative decisioning, no hard-coded branching.

Q43
Scenario: An assignment must be worked by exactly one of several skilled operators, and if none respond in time, widen the eligible pool.

Question: **How do you implement skill-based routing with progressive fallback?**

> **Ideal answer focus:** Skill-based routing to a work queue filtered by skills; SLA passed-deadline escalation that broadens the skill filter or reroutes to a wider queue; keep routing in when/decision rules; avoid operator-ID hard-coding.

Q44
Scenario: A regulator requires that certain case actions capture an explicit reason and second-person sign-off before proceeding.

Question: **How do you enforce reason capture and dual control in the lifecycle?**

> **Ideal answer focus:** Flow action collecting a mandatory reason (validation); a separate approval assignment routed to a different operator (segregation of duties); privileges; audit via case history; do not rely on UI-only enforcement.

Q45
Scenario: Business wants a 'save and complete later' capability so users can pause data entry on a long form and resume days later.

Question: **How do you support draft/resume without breaking the flow?**

> **Ideal answer focus:** Persist the case at the assignment (Save action) so it stays on the worklist; resume the same assignment later; ensure validation runs on submit, not on save; consider a draft status; case locking on resume.

Q46
Scenario: Cases created via an inbound channel must be automatically categorized and routed differently from web-created ones.

Question: **How do you drive origin-aware intake behavior?**

> **Ideal answer focus:** Capture the origin/channel (x-origin-channel or a source property); decision-driven categorization and routing; circumstance where behavior differs by channel; keep the microjourney/logic shared.

Q47
Scenario: A parent case must show a live rollup of child case statuses and totals on its summary view.

Question: **How do you aggregate child data onto the parent efficiently?**

> **Ideal answer focus:** Report/data page over child cases keyed by parent; declarative rollup where feasible; refresh on child status change (Declare Trigger/OnChange); avoid expensive synchronous recalculation on every open; expose needed fields.

Q48
Scenario: A flow uses many nested subprocesses; when something fails, it's hard to know which subprocess caused it.

Question: **How do you design for traceability and error isolation in nested processes?**

> **Ideal answer focus:** Keep subprocesses focused and named meaningfully; add flow-level error handling (FlowProblems / exception routing) per subprocess; use case history and Tracer; avoid excessive nesting; surface errors to a review path.

Q49
Scenario: An SLA must start only when a dependency (document received) is met, not at assignment creation.

Question: **How do you defer SLA start until a condition holds?**

> **Ideal answer focus:** Attach the SLA at the point/step where the dependency is satisfied, or gate with a Wait step until the condition; consider a case-level SLA starting at a specific stage; avoid a timer counting during a legitimate wait.

Q50
Scenario: A business wants to reassign all in-flight work from a decommissioned team to a new team overnight.

Question: **How do you perform bulk reassignment safely and support ongoing routing?**

> **Ideal answer focus:** Bulk transfer/reassign via Case Manager tooling or a controlled job; update routing rules (work queue/skills) so new work goes to the new team; test; audit; avoid manual per-case moves at scale.

Q51
Scenario: A case must enforce that once it enters 'Underwriting', earlier-stage data becomes read-only.

Question: **How do you make prior-stage data immutable after progression?**

> **Ideal answer focus:** Field visibility/edit conditions keyed on current stage/status (when rules); server-side validation to reject edits; in Constellation, editability is view metadata driven by conditions; don't rely on UI alone — enforce on save.

Q52
Scenario: Stakeholders want to measure cycle time per stage to find bottlenecks.

Question: **How do you instrument the lifecycle for stage-level analytics?**

> **Ideal answer focus:** Leverage stage entry/exit timestamps and case history; expose/optimize timing fields; report via Insights; consider case performance/analytics features; design statuses/stages to be measurable.

Q53
Scenario: A complex case occasionally needs a human to manually move it to a different stage to correct a mistake.

Question: **How do you allow controlled manual stage changes?**

> **Ideal answer focus:** Change-stage optional action guarded by a privilege (supervisor); audit the change; validate the target is legal; avoid ad-hoc ticket abuse; ensure declarative/SLA reset behaves correctly on the jump.

Q54
Scenario: A case type shared across brands needs brand-specific correspondence, SLAs, and approval limits.

Question: **How do you specialize multiple aspects per brand cleanly?**

> **Ideal answer focus:** Circumstance correspondence/SLA/decision rules by brand; externalize limits to application settings; keep base case type and flows shared; use ECS layering if brands are large; relevant records for reuse.

Q55
Scenario: Users report that submitting an assignment sometimes double-creates a follow-up child case.

Question: **How do you prevent duplicate side effects from re-submission or retries?**

> **Ideal answer focus:** Idempotency: guard child creation with a when check (not already created) or a flag; understand double-submit/retry causes; in Constellation, handle eTag/optimistic-lock retries safely; avoid unconditional create steps.

Q56
Scenario: A long-lived case must survive an application upgrade mid-lifecycle without breaking in-flight flows.

Question: **What design and upgrade practices protect in-flight cases?**

> **Ideal answer focus:** Avoid breaking flow changes on active cases; use flow versioning considerations; keep rules backward compatible; test upgrade with in-flight data; ruleset versioning; don't delete rules that active cases reference.

Q57
Scenario: A case must automatically create a corrective sub-case whenever a quality check fails, and link it back for tracking.

Question: **How do you model exception-triggered child cases with traceability?**

> **Ideal answer focus:** Decision on the QC result creates a linked child case (create-case automation); parent-child relationship for tracking; propagate context; parent may wait or continue; report rollup; avoid manual creation.

Q58
Scenario: An intake form must adapt its steps based on answers given earlier in the same case.

Question: **How do you build dynamic, answer-driven process paths?**

> **Ideal answer focus:** Decision-driven process/stage selection using earlier case data (when/decision rules); conditionally skip processes/stages; keep branching declarative; in Constellation the view sequence follows the case design.

Q59
Scenario: A time-critical case must be resolved within an hour or it is auto-cancelled and the customer notified.

Question: **How do you implement a hard time limit with automatic terminal action?**

> **Ideal answer focus:** Case/assignment SLA with a passed-deadline action that sets Resolved-Cancelled and sends correspondence; ticket to jump to a cancel end; ensure the timer/cancel is idempotent; audit.

Q60
Scenario: Business wants a supervisor dashboard of all cases breaching or near-breaching SLA.

Question: **How do you surface SLA risk operationally?**

> **Ideal answer focus:** Insights/report on urgency, goal/deadline timestamps, and status; expose timing properties; near-breach filter; supervisor portal widget; refresh strategy; drive from optimized fields.

Q61
Scenario: A multi-party case (customer, agent, underwriter) needs role-specific views and actions on the same case.

Question: **How do you tailor experience per participant while sharing one case?**

> **Ideal answer focus:** Work parties for participants; persona/role-based Views and action availability; security controls; same case, different rendered Views by persona; keep logic shared, presentation specialized.

Q62
Scenario: A case type must comply with a rule that certain steps happen in a legally mandated order, never skipped.

Question: **How do you enforce a non-bypassable ordered sequence?**

> **Ideal answer focus:** Sequential steps with server-side validation/guards preventing skip; no optional bypass on mandated steps; audit; avoid tickets/change-stage that could jump past mandatory work; test edge cases.

Q63
Scenario: A high-volume case type creates worklist clutter; agents should only see what they can act on now.

Question: **How do you keep worklists focused and performant?**

> **Ideal answer focus:** Route only actionable assignments to worklists; use workbaskets + Get Next Work; filter by skills; avoid parking non-actionable items on worklists; optimize the underlying queries.

Q64
Scenario: A case must branch to different resolution paths based on a score computed from many inputs.

Question: **How do you compute and act on a composite score in the lifecycle?**

> **Ideal answer focus:** Declare Expression / decision to compute the score; decision table/tree to route on score bands; keep the scoring configurable; document; test boundary bands including default.

Q65
Scenario: Stakeholders want to pilot a new stage design for 10% of cases before full rollout.

Question: **How do you controlled-release a lifecycle change?**

> **Ideal answer focus:** Circumstance/route a subset by a pilot flag; branch ruleset for the new design; feature toggle via application setting; measure with Insights; promote when validated; keep rollback simple.

Q66
Scenario: A case reopened for correction must not re-run integrations that already succeeded.

Question: **How do you make re-entrant lifecycle steps safe?**

> **Ideal answer focus:** Guard integration/automation steps with when conditions (already-done flags); idempotency; track completion state on the case; test the reopen path; avoid unconditional re-execution.

Q67
Scenario: An approval chain must collapse to a single approver when the requester is themselves a senior manager.

Question: **How do you make approval depth context-dependent?**

> **Ideal answer focus:** Cascading approval driven by a decision considering requester level; skip levels the requester already satisfies; segregation-of-duties check; configurable authority matrix; audit the resolved chain.

Q68
Scenario: Business wants to attach and require specific documents at defined points, blocking progress until provided.

Question: **How do you enforce document requirements in the flow?**

> **Ideal answer focus:** Attachment requirements at the step with validation gating submission; Wait for required attachments; use attachment categories; server-side enforcement; user guidance; store attachments externally at scale.

Q69
Scenario: A case must notify different stakeholders at each stage transition with tailored content.

Question: **How do you drive stage-based, audience-specific communications?**

> **Ideal answer focus:** Send-email automations on stage entry using circumstanced/parameterized correspondence per audience; when conditions to avoid duplicate sends on reopen; keep templates reusable and localizable.

Q70
Scenario: An operational team needs to hand off a case between shifts without losing context or progress.

Question: **How do you support seamless case handoff?**

> **Ideal answer focus:** Save-and-resume on the assignment; reassign to the next shift's work queue; case history and Pulse/notes for context; SLAs continue; avoid data loss on handoff; locking on resume.

Q71
Scenario: A case type must support both English and Spanish users concurrently with correct labels and correspondence.

Question: **How do you deliver a bilingual lifecycle experience?**

> **Ideal answer focus:** Localization: language packs for labels/messages, localized correspondence; locale-driven rendering; keep text externalized; test both locales; correspondence circumstanced/localized by locale.

Q72
Scenario: A case must support bulk actions where a manager approves 50 assignments at once.

Question: **How do you implement performant bulk case actions?**

> **Ideal answer focus:** Bulk processing action; queue heavy work asynchronously (queue processor) rather than 50 synchronous submits; per-item error handling; progress feedback; avoid blocking the request/DX call.

Q73
Scenario: A stage must run different processes depending on the customer segment determined at runtime.

Question: **How do you select processes dynamically within a stage?**

> **Ideal answer focus:** Decision-driven process selection (when/decision table) at stage entry; conditionally skip alternatives; keep segment logic declarative and configurable; reuse shared subprocesses.

Q74
Scenario: A case's flow must retry a failed automated step a few times before routing to manual handling.

Question: **How do you build bounded retry with fallback in a flow?**

> **Ideal answer focus:** Retry via queue processor with a retry limit/backoff, or a loop with a counter and guard; after N failures route to a manual review assignment; idempotency; surface the error; audit attempts.

Q75
Scenario: Business wants to prevent two conflicting optional actions from being run simultaneously on a case.

Question: **How do you prevent conflicting concurrent actions?**

> **Ideal answer focus:** Case locking (optimistic eTag in Constellation) rejects the stale second action; guard actions with status/when conditions; design mutually-exclusive availability; handle conflict gracefully in the UI.

Q76
Scenario: A regulatory change requires adding a mandatory step to all in-flight and future cases of a type.

Question: **How do you introduce a mandatory step affecting active cases safely?**

> **Ideal answer focus:** Add the step with careful flow versioning; ensure in-flight cases route through it (ticket/flow update strategy); test with active-case data; avoid breaking cases already past that point; communicate/roll out controlled.

Q77
Scenario: A case must expose a read-only 'case summary' consumable by an external dashboard.

Question: **How do you expose case data for external consumption cleanly?**

> **Ideal answer focus:** DX API case/view endpoints or a purpose-built data view/service; OAuth-secured; expose only needed fields; read-only; keep logic server-side; DX compliance.

Q78
Scenario: Users need in-context collaboration (comments, @mentions) on a case without email back-and-forth.

Question: **How do you enable collaboration within the case?**

> **Ideal answer focus:** Pulse/collaboration widgets and tags; @mentions and notifications; keep discussion attached to the case history; reduces external email; secure by case access.

Q79
Scenario: A case type must auto-populate defaults differently based on how the case was created.

Question: **How do you vary case initialization by creation context?**

> **Ideal answer focus:** Data transform (pyDefault) with when-conditions on the creation source/channel; circumstance where large differences exist; keep initialization declarative; test each entry point.

Q80
Scenario: A supervisor needs to pull a specific case out of a queue and assign it to a named specialist urgently.

Question: **How do you support targeted manual assignment overrides?**

> **Ideal answer focus:** Reassign/transfer action (privileged) moving the assignment to the specialist's worklist; bump urgency; audit; ensure routing rules don't immediately re-route it; Case Manager tooling.

Q81
Scenario: A case's SLA should pause while waiting on the customer and resume when they respond.

Question: **How do you implement pause/resume of SLA timing?**

> **Ideal answer focus:** Model the customer-wait as a Wait/assignment where the SLA is not counting against the team (separate SLA or paused timer); resume team SLA on response; design timing so waits on external parties don't breach internal SLAs.

Q82
Scenario: Business wants to test whether reordering two stages improves conversion.

Question: **How do you experiment with lifecycle structure and measure outcomes?**

> **Ideal answer focus:** Branch/circumstance the alternate stage order for a cohort; capture outcome metrics via Insights; compare; promote the winner; keep changes reversible; data-driven.

Q83
Scenario: A case type is used globally and must respect regional compliance differences in its steps.

Question: **How do you handle region-specific compliance within one case type?**

> **Ideal answer focus:** Circumstance compliance-related steps/rules by region; conditionally include region-only stages; externalize region rules to decision tables/settings; keep the core microjourney shared; audit.

Q84
Scenario: An assignment must be automatically completed by the system when an external confirmation arrives.

Question: **How do you auto-complete a human assignment via an external event?**

> **Ideal answer focus:** Inbound service/callback updates the case and advances the assignment (e.g., via an activity/queue processor performing the flow action) or raises a ticket; idempotency; avoid a stuck manual assignment; audit the automated action.

Pillar B — Data Modelling & Integration

Q85
Scenario: A reference list of 50,000 products is loaded per user session, spiking memory and slowing login for thousands of concurrent users.

Question: **How do you choose the correct data page scope and loading strategy here?**

> **Ideal answer focus:** Node-level read-only data page (shared across users, loaded once per node) instead of requestor/thread scope; consider keyed data page + lookup instead of loading all 50k; reload strategy; avoid caching huge lists per requestor. Guardrail: right-size data page scope.

Q86
Scenario: A REST integration returns 30 fields but the app uses 5. The full payload is mapped into the case and bloats the BLOB.

Question: **How do you design efficient response mapping?**

> **Ideal answer focus:** Map only needed fields into a data page/data object (not the case BLOB); use response data transform to project required properties; keep integration decoupled behind a data page; expose/optimize only reportable fields.

Q87
Scenario: A customer lookup calls an external CRM on every keystroke in a type-ahead, hammering the CRM and hitting rate limits.

Question: **How do you design the data sourcing to protect the source system?**

> **Ideal answer focus:** Data page with appropriate caching/scope (node-level for reference; keyed for entity), debounce on the client, paginate; consider a search service endpoint; reload-on-condition rather than per-keystroke; respect rate limits.

Q88
Scenario: An external service is sometimes down; when it is, the whole case creation fails with a stack trace shown to users.

Question: **How do you build graceful degradation for integration failures?**

> **Ideal answer focus:** Data page error handling (onError), connector fault handling, fallback/default data, retry via queue processor, user-friendly messaging; decouple UI from the connector via the data page so the app degrades instead of crashing. Guardrail: robust error handling.

Q89
Scenario: You must integrate with a legacy SOAP service (WSDL) and a modern REST API, and present a unified customer object.

Question: **How do you architect a multi-source unified data model?**

> **Ideal answer focus:** Connect-SOAP (WSDL) and Connect-REST behind separate data pages; combine into one data object/data page the app references; map both into a common structure; keep systems-of-record decoupled; authentication profiles per connector.

Q90
Scenario: Real-time inventory must be reflected in the UI within seconds of changing in the source system.

Question: **How do you achieve near-real-time data without constant polling?**

> **Ideal answer focus:** Consider event-driven ingestion (Kafka/stream, service push updating Pega), data page reload-on-condition, short cache with keyed data page; or push via service that updates the case; discuss trade-offs of polling vs push.

Q91
Scenario: A savable data page must write updates back to the CRM, but partial failures leave Pega and CRM out of sync.

Question: **How do you design reliable write-back with a savable data page?**

> **Ideal answer focus:** Savable data page save plan calling the connector; transaction/commit boundaries; error handling and compensation; idempotent writes; confirm success before updating local state; consider queueing for retry.

Q92
Scenario: Integration credentials (OAuth tokens, API keys) were hard-coded in a connector and later exported in a RAP, exposing secrets.

Question: **How should integration credentials be managed?**

> **Ideal answer focus:** Authentication profiles referencing a keystore; OAuth 2.0 profiles; never store secrets in rules; environment-specific via dynamic system settings; secrets excluded from packaging. Guardrail: secure credential handling.

Q93
Scenario: The same integration endpoint differs across dev/test/prod, and promotions keep breaking because URLs are baked into rules.

Question: **How do you make integrations environment-portable?**

> **Ideal answer focus:** Dynamic System Settings / application settings for endpoints; reference from connector; no hard-coded URLs; keep one rule, different values per environment; Deployment Manager promotes rules, settings stay per-env.

Q94
Scenario: A data page keyed by customer ID caches stale data after the customer's record is updated elsewhere.

Question: **How do you control data page freshness for keyed entities?**

> **Ideal answer focus:** Reload-on-condition tied to an update event/timestamp; short expiry; explicit refresh after known updates; balance freshness vs source load; avoid indefinite caching of mutable entity data.

Q95
Scenario: A batch of 100k records must be imported and validated nightly without impacting daytime users.

Question: **How do you architect high-volume batch ingestion?**

> **Ideal answer focus:** Job Scheduler (replacing legacy agents) or queue processor for async batch; chunk/stream processing; validate and log errors to a report; off-peak scheduling; monitor via Admin Studio; avoid loading all into memory.

Q96
Scenario: A REST connector works in Postman but fails in Pega with SSL/handshake errors.

Question: **How do you troubleshoot connector-level integration failures?**

> **Ideal answer focus:** Check the keystore/truststore and certificates, authentication profile, endpoint URL, headers, and use Connector & Metadata / Tracer + integration logs; verify TLS version; test with the connector's 'Run' and simulate; inspect the actual request/response.

Q97
Scenario: Response JSON structure changes intermittently (a field is sometimes an object, sometimes an array), breaking mapping.

Question: **How do you build resilient mapping against variable payloads?**

> **Ideal answer focus:** Robust JSON data mapping / data transform with conditional handling; tolerant parsing; validate response shape; version the integration; coordinate contract with the provider; avoid brittle positional mapping.

Q98
Scenario: A data page must be parameterized by multiple keys (region + product) and reused across many rules.

Question: **How do you design a multi-parameter keyed data page?**

> **Ideal answer focus:** Define parameters and keys on the data page; reference as D_Page[region, product]; ensures per-key caching; document parameters; reuse via relevant records; avoid duplicating similar data pages.

Q99
Scenario: The team debates whether a 'Customer' should be a case type, a data object, or a data page-backed reference.

Question: **How do you decide the right modelling construct for an entity?**

> **Ideal answer focus:** Case type if it has a lifecycle/outcome; data object for a reusable entity structure; data page for sourcing/caching from a system of record; combine (data object modelled, data page sourced). Center-out data layer separation.

Q100
Scenario: An embedded page list of order lines cannot be reported on or searched.

Question: **How do you make embedded/list data queryable?**

> **Ideal answer focus:** Declare Index to expose embedded list into an index table; or model as child cases/separate class if heavily queried; expose/optimize properties; report via Insights on the exposed data.

Q101
Scenario: A property used heavily in report filters and Get-Next-Work is stored only in the BLOB, making queries slow.

Question: **Why is this slow and how do you fix it?**

> **Ideal answer focus:** BLOB properties aren't directly queryable; expose/optimize the property to a dedicated column (Property Optimization); rebuild index; verify DB column + index; measure with DB Trace/PAL. Guardrail: optimize reportable properties.

Q102
Scenario: You must call an external service that requires a signed JWT and rotating keys.

Question: **How do you handle advanced authentication in Pega integrations?**

> **Ideal answer focus:** OAuth 2.0 / JWT via authentication profile; keystore for keys; token caching and refresh; custom auth where needed but prefer configured profiles; secure storage; rotate via keystore updates.

Q103
Scenario: Data from three services must be aggregated, but one is slow and blocks the whole screen load.

Question: **How do you prevent a slow source from degrading the whole UX?**

> **Ideal answer focus:** Load independent data pages asynchronously/lazily; defer the slow one (load on demand / separate view); set timeouts; progressive rendering; in Constellation, DX API returns per-view data so structure views to isolate the slow source.

Q104
Scenario: A client insists on storing full external documents inside the case, ballooning storage.

Question: **How do you handle large attachments/documents at scale?**

> **Ideal answer focus:** Store attachments in external content storage (attachment storage / repository / cloud), reference by link; avoid embedding large binaries in the case BLOB; use Pega attachment management; retention policy.

Q105
Scenario: A data transform copying source-to-target is being replaced by an activity 'for flexibility'.

Question: **Why prefer a data transform, and when is an activity justified?**

> **Ideal answer focus:** Data transform is declarative, maintainable, guardrail-compliant, supports pageInstructions for Constellation embedded data; activity only for logic configuration can't express; keep data mapping in transforms. Guardrail: minimize activities.

Q106
Scenario: In Constellation, updating an embedded page list via DX API isn't persisting correctly.

Question: **How do you correctly update embedded/list data through the DX API?**

> **Ideal answer focus:** Use pageInstructions in the request body to insert/update/delete embedded page list/group entries; content for top-level fields; the DX API requires pageInstructions for nested lists rather than direct assignment. Cite request-body structure.

Q107
Scenario: A savable data page write must be atomic with the case save, but currently they commit separately.

Question: **How do you coordinate transactional consistency between case and external data?**

> **Ideal answer focus:** Understand deferred save vs commit; coordinate commit boundaries; use save plans; where cross-system atomicity is impossible, design compensation/retry and idempotency; document eventual consistency.

Q108
Scenario: An integration returns 10k rows but the UI only needs the current page.

Question: **How do you design pagination for large result sets?**

> **Ideal answer focus:** Server-side pagination via the source/report; keyed/paged data page; DX API data view endpoints with paging query params; avoid loading all rows into the clipboard; lazy-load on scroll.

Q109
Scenario: Two data pages load overlapping data, causing inconsistent values on the same screen.

Question: **How do you ensure a single source of truth for shared data?**

> **Ideal answer focus:** Consolidate to one canonical data page/data object; reference it everywhere; avoid duplicate sources; scope appropriately; document ownership. Guardrail: single source of truth.

Q110
Scenario: A connector must post a complex nested request built from case data.

Question: **How do you construct and validate a complex outbound request?**

> **Ideal answer focus:** Map via data transform into the request data model; use the connector's request mapping; validate before send; test with connector Run/simulation; handle response and faults; keep mapping declarative.

Q111
Scenario: The business wants to expose a Pega data view to an external portal securely.

Question: **How do you expose Pega data via the DX API to a third party?**

> **Ideal answer focus:** DX API data view endpoints (read/savable data pages exposed as data views); OAuth-secured; view-based; ensure DX API compliance; control which data views are API-accessible; rate limiting.

Q112
Scenario: A reference data page rarely changes but is critical; on node restart the first user pays a huge load cost.

Question: **How do you warm/preload critical reference data?**

> **Ideal answer focus:** Pre-load node-level data pages at startup (load management / pre-loading), or scheduled refresh; accept lazy load with acceptable first-hit cost; monitor; avoid per-user reload.

Q113
Scenario: An external API enforces strict rate limits; bursts of case activity trigger throttling and failures.

Question: **How do you smooth outbound call volume?**

> **Ideal answer focus:** Queue outbound calls via queue processor with controlled concurrency/rate; cache with data pages to reduce calls; batch where possible; backoff/retry on 429; monitor.

Q114
Scenario: Data quality from the source is poor (nulls, bad formats), causing downstream rule failures.

Question: **How do you defend the application against dirty inbound data?**

> **Ideal answer focus:** Validate/cleanse on ingestion (data transform + validate rules); default/normalize; reject or quarantine bad records; log to an error report; don't let bad data propagate into cases.

Q115
Scenario: A mobile channel needs a trimmed data payload while web needs full data, from the same case.

Question: **How do you tailor data returned per channel efficiently?**

> **Ideal answer focus:** Different Views per channel (circumstanced by x-origin-channel), DX API returns only the view's fields; viewType controls metadata volume; keep business logic shared; avoid over-fetching on mobile.

Q116
Scenario: The team is unsure whether to use DX API v1 (traditional) or Constellation DX API for a new front end.

Question: **How do you decide, and what are the implications?**

> **Ideal answer focus:** New Constellation apps use Constellation DX API v2 (view-based, section/harness-free); v1 is for legacy section-based UI-Kit/Theme Cosmos; mixing is limited; choose v2 for greenfield; ensure app is DX-API compliant.

Q117
Scenario: A migration from an on-prem DB integration to a cloud API must happen with zero downtime.

Question: **How do you migrate an integration behind a data layer without disrupting the app?**

> **Ideal answer focus:** Because UI/process reference the data page (not the source), swap the data page's source from DB to API; keep the interface stable; feature-flag/settings-driven cutover; test in parallel. This is the value of decoupling via data pages.

Q118
Scenario: Real-time mapping must transform a partner's field names/units into the app's canonical model consistently.

Question: **How do you centralize and reuse transformation logic?**

> **Ideal answer focus:** Canonical data object + reusable data transforms for mapping/normalization; keep transformation in one place; version it; apply on inbound and outbound; avoid scattering mapping logic.

Q119
Scenario: A data page's parameters are user-supplied and could be manipulated to fetch unauthorized data.

Question: **How do you secure parameterized data access?**

> **Ideal answer focus:** Validate/authorize parameters server-side; enforce access control policies on the underlying class; don't trust client-supplied keys blindly; scope data by operator context; audit.

Q120
Scenario: You need to test integrations before the partner's endpoint is ready.

Question: **How do you develop and test integrations independently?**

> **Ideal answer focus:** Simulations / stub data sources on the data page/connector; mock responses; contract-first design against the agreed schema; switch to live via settings; enables parallel development.

Q121
Scenario: A connector's response time varies wildly and you need to know the true impact on user experience.

Question: **How do you measure and diagnose integration performance?**

> **Ideal answer focus:** PAL (Performance Analyzer) for elapsed/connector time, Connector & Metadata Trace / integration logs, DB Trace where relevant; identify slow calls; add caching/async; set SLAs/timeouts; monitor with PDC.

Q122
Scenario: Business wants offline/eventual updates from a mobile app synced back to Pega cases.

Question: **How do you architect intermittent-connectivity data sync?**

> **Ideal answer focus:** Offline-capable design (mobile SDK/offline), queue changes, sync via DX API when online; conflict handling with eTag/optimistic locking; idempotent writes; reconcile.

Q123
Scenario: A single logical 'address' is stored differently in five systems.

Question: **How do you present a consistent address while integrating heterogeneous sources?**

> **Ideal answer focus:** Canonical Address data object; per-source data pages + mapping transforms into the canonical form; reference the canonical everywhere; hide source differences behind the data layer.

Q124
Scenario: A data page sourced from a report definition returns thousands of rows and slows every screen that references it.

Question: **How do you design the data page + report for scale?**

> **Ideal answer focus:** Filter/paginate at the report level; return only needed columns (optimized); keyed/paged data page; appropriate scope; avoid returning full datasets to the clipboard; measure with DB Trace/PAL.

Q125
Scenario: An external system requires requests in a specific XML schema, but the app's data model is flat.

Question: **How do you map between a rich external schema and the internal model?**

> **Ideal answer focus:** Reusable data transforms / XML mapping in the connector; canonical internal data object; keep mapping declarative and centralized; validate against schema; version the mapping.

Q126
Scenario: A partner webhook posts events to Pega that must create or update the correct case within seconds.

Question: **How do you architect inbound event handling reliably?**

> **Ideal answer focus:** Inbound Service REST in a service package (OAuth-secured); map payload; queue for async processing (queue processor) to decouple ingestion from processing; idempotency by event ID; error/DLQ handling; fast acknowledgment.

Q127
Scenario: The same reference data must be available offline in a mobile scenario.

Question: **How do you provide reference data to offline clients?**

> **Ideal answer focus:** Sync reference data to the device (mobile/offline capability); keyed data pages; periodic sync; conflict handling on write-back; keep payloads lean; DX API for online refresh.

Q128
Scenario: A data page's source connector fails; the app should fall back to cached or default values rather than error.

Question: **How do you build layered fallback for data sourcing?**

> **Ideal answer focus:** Data page error handling with fallback source/default; last-known-good caching; user messaging; circuit-breaker style avoidance of repeated failing calls; degrade gracefully. Guardrail: never surface raw stack traces.

Q129
Scenario: You must integrate with a system that only supports file-based exchange (SFTP drop).

Question: **How do you architect file-based integration in Pega?**

> **Ideal answer focus:** File listener / File data set / repository integration; scheduled pickup (Job Scheduler); parse and map; validate and log errors; archive processed files; async processing for large files.

Q130
Scenario: A REST API is paginated with cursors; you must retrieve all pages to build a complete list.

Question: **How do you handle cursor/page-based aggregation without blocking users?**

> **Ideal answer focus:** Async retrieval (queue processor) looping through cursors; assemble into a data set/data page; avoid synchronous multi-call loops in the UI thread; cache the assembled result; respect rate limits.

Q131
Scenario: Multiple applications need the same integration; each team is building its own connector.

Question: **How do you promote integration reuse across an enterprise?**

> **Ideal answer focus:** Shared integration/data layer in a reuse ruleset/layer (ECS); expose reusable data pages/data objects; relevant records; a common canonical model; avoid duplicate connectors; governance.

Q132
Scenario: A data object's structure changed in the source, and now mappings silently drop fields.

Question: **How do you manage integration contract/schema evolution?**

> **Ideal answer focus:** Versioned integrations and mappings; validate response contract; monitor for missing fields; coordinate with the provider; add tests; avoid brittle mapping that fails silently.

Q133
Scenario: Sensitive PII flows through an integration and must not be logged or cached longer than necessary.

Question: **How do you handle PII in the data/integration layer?**

> **Ideal answer focus:** Avoid logging PII (mask in traces/logs); minimal caching with short expiry and appropriate scope; encryption; access control policies on the data class; comply with retention; ZDR considerations.

Q134
Scenario: A savable data page must upsert (insert or update) into a system of record based on a natural key.

Question: **How do you design upsert semantics reliably?**

> **Ideal answer focus:** Save plan logic checking existence by key then insert/update; idempotency; handle race conditions; confirm success; error handling; keep it in the save plan rather than scattered activities.

Q135
Scenario: The team wants to validate that the app is DX-API compliant so a custom front end can consume its data.

Question: **How do you ensure and verify DX API compliance for the data/UI layer?**

> **Ideal answer focus:** View-based authoring, screen layout as harness container, supported components only; run DX API compliance checks; avoid unsupported/custom controls; ensure data views are properly exposed.

Q136
Scenario: An integration returns dates/times in the source's timezone, causing wrong values for users.

Question: **How do you handle timezone and locale correctness across integration boundaries?**

> **Ideal answer focus:** Normalize to a canonical timezone (UTC) on ingestion; store consistently; render in user locale/timezone via Pega localization; document assumptions; test across zones.

Q137
Scenario: A high-frequency read (pricing) and a low-frequency write (config) touch the same data object.

Question: **How do you optimize caching for mixed read/write access patterns?**

> **Ideal answer focus:** Read via a cached read-only data page (node-level) with reload-on-condition; writes via a savable data page that triggers cache refresh; separate concerns; avoid caching volatile write data indefinitely.

Q138
Scenario: You must combine data from Pega and two external systems into one report/insight.

Question: **How do you report across federated data sources?**

> **Ideal answer focus:** Materialize/index external data where needed (data set / declare index / persisted), or use report joins on exposed data; consider periodic sync into reportable classes; Insights over optimized data; avoid live cross-system joins at query time.

Q139
Scenario: A data object must be sourced live for display but persisted into the case only at a specific commit point.

Question: **How do you separate referenced (live) data from persisted case data?**

> **Ideal answer focus:** Reference via read-only data page for display; copy into the case (data transform) only when the business requires a snapshot at commit; understand reference vs copy trade-offs; avoid premature persistence.

Q140
Scenario: An API returns HTTP 200 with an error object in the body instead of a proper error status.

Question: **How do you detect and handle 'soft' errors in responses?**

> **Ideal answer focus:** Inspect response body for error indicators (not just HTTP status); map to fault handling; fail the data page/connector explicitly; route case to exception path; don't treat 200-with-error as success.

Q141
Scenario: A large data page is shared node-level, but different users must see row-level-filtered subsets.

Question: **How do you combine shared caching with per-user filtering?**

> **Ideal answer focus:** Cache the full reference set node-level; apply per-user filtering at access time (access control policy / filtered view) rather than per-user caching; avoid duplicating caches per user; secure server-side.

Q142
Scenario: You must expose a savable data page as an updatable data view to a custom Constellation front end.

Question: **How do you enable API-driven writes via a data view?**

> **Ideal answer focus:** Savable data page exposed as a data view through the DX API; OAuth-secured; validate/authorize writes server-side; idempotent save plan; ensure DX compliance; control which data views are writable.

Q143
Scenario: A connector call must include a correlation ID for end-to-end tracing across systems.

Question: **How do you implement distributed traceability in integrations?**

> **Ideal answer focus:** Generate/propagate a correlation ID in request headers (parameterized connector); log it (without PII); align with the partner's tracing; aids cross-system diagnosis; store on the case for support.

Q144
Scenario: Reference data changes must invalidate caches across all nodes near-instantly.

Question: **How do you handle cluster-wide cache invalidation for data pages?**

> **Ideal answer focus:** Reload-on-condition tied to a change signal; cluster cache management; short expiry as a fallback; event/notification-driven refresh; accept eventual consistency window; monitor.

Q145
Scenario: A batch integration sometimes partially completes; you need to resume from the failure point.

Question: **How do you design restartable batch processing?**

> **Ideal answer focus:** Checkpointing/track processed keys; idempotent per-item processing via queue processor; skip already-processed items on restart; error report for failures; avoid reprocessing successes.

Q146
Scenario: An outbound call must be fire-and-forget from the user's perspective but guaranteed to eventually run.

Question: **How do you decouple user interaction from guaranteed delivery?**

> **Ideal answer focus:** Queue the call via a queue processor (durable, at-least-once); return control to the user immediately; retries/DLQ for reliability; idempotency at the endpoint; monitor the queue.

Q147
Scenario: A data model must support both a canonical enterprise object and app-specific extensions.

Question: **How do you layer shared and specific data structures?**

> **Ideal answer focus:** Canonical data object in a reuse layer; extend/specialize per app via inheritance or additional properties; keep the shared contract stable; ECS layering; avoid forking the canonical model.

Q148
Scenario: A report must show current values from an external system, not a stale copy stored in the case.

Question: **How do you report on live external data?**

> **Ideal answer focus:** Reference external data via a data page at view time rather than reporting stale case copies; or periodically sync into a reportable class; choose based on freshness vs performance; Insights over the chosen source.

Q149
Scenario: A connector's payload occasionally exceeds size limits and is rejected.

Question: **How do you handle oversized integration payloads?**

> **Ideal answer focus:** Chunk/paginate the payload; compress where supported; send references instead of inline blobs; async processing; validate size before send; coordinate limits with the provider.

Q150
Scenario: You must integrate with a system requiring mutual TLS (client certificates).

Question: **How do you configure mTLS integrations in Pega?**

> **Ideal answer focus:** Client keystore with the client certificate + truststore for the server CA; reference from the connector/auth profile; manage certificate rotation; secure key storage; test the handshake.

Q151
Scenario: A data page must be preloaded with parameters derived from the logged-in operator.

Question: **How do you source operator-contextual data safely?**

> **Ideal answer focus:** Parameterize the data page with operator context (from the operator/requestor page); load per requestor scope; authorize server-side; avoid trusting client-supplied identity; keep secure.

Q152
Scenario: Two integrations must be called in sequence where the second depends on the first's output.

Question: **How do you orchestrate dependent integration calls cleanly?**

> **Ideal answer focus:** Sequence via data pages/data transforms or an integration process; first call populates data used by the second; handle first-call failure before the second; keep orchestration declarative; async if long-running.

Q153
Scenario: Business wants to switch an integration provider with a feature flag and roll back instantly if issues arise.

Question: **How do you design provider-swappable integrations?**

> **Ideal answer focus:** Abstract behind a data page/data object; select the provider connector via application setting/decision; feature-flag cutover; parallel-run/validate; instant rollback by flipping the setting; the data layer decoupling enables this.

Q154
Scenario: A data page must aggregate paginated results but only load additional pages as the user scrolls.

Question: **How do you implement lazy/incremental data loading?**

> **Ideal answer focus:** Paged data view via DX API with page params; load-on-scroll in the client; keyed paging on the source; avoid loading all rows; cache loaded pages; keep clipboard lean.

Q155
Scenario: An integration must transform between the app's canonical model and three different partner formats.

Question: **How do you manage many-to-one/one-to-many format mapping maintainably?**

> **Ideal answer focus:** Canonical data object + a reusable mapping transform per partner; single canonical hub; version each mapping; keep partner specifics isolated; avoid partner logic leaking into core rules.

Q156
Scenario: A high-value write to an external system must be confirmed and reconciled if the response is lost.

Question: **How do you handle uncertain outcomes on critical writes?**

> **Ideal answer focus:** Idempotent write + reconciliation query to confirm state; retry safely; store a pending status until confirmed; compensation if failed; avoid double-writes; audit.

Q157
Scenario: A data page must be shared but personalized per user's entitlements.

Question: **How do you serve shared-but-filtered reference data securely?**

> **Ideal answer focus:** Cache shared node-level; filter at access via access control policy/entitlement check; don't cache per user; enforce server-side; keep the shared cache authoritative.

Q158
Scenario: You must ingest streaming events at high volume and update cases accordingly.

Question: **How do you architect high-throughput event ingestion?**

> **Ideal answer focus:** Stream/Kafka data set + queue processor consumers; async, partitioned, idempotent processing; backpressure handling; monitor lag; decouple ingestion from case updates; scale consumers.

Q159
Scenario: A REST endpoint requires OAuth with a token that must be cached and refreshed centrally.

Question: **How do you manage token lifecycle for many concurrent calls?**

> **Ideal answer focus:** OAuth 2.0 authentication profile handling token acquisition/refresh/caching centrally; keystore for secrets; avoid per-call token requests; handle refresh on expiry; reuse across connectors.

Q160
Scenario: A data model must represent a hierarchy (org units, categories) of arbitrary depth.

Question: **How do you model and traverse hierarchical data?**

> **Ideal answer focus:** Self-referencing data object / parent-key structure; source via data page; traverse via reports/declare index or recursive logic where needed; expose keys for querying; consider performance of deep traversal.

Q161
Scenario: An integration's data must be masked in non-production environments for privacy.

Question: **How do you handle sensitive data across environments?**

> **Ideal answer focus:** Data masking/anonymization for lower environments; environment-specific sources (settings); avoid copying real PII to test; access controls; comply with privacy; use synthetic/simulated data.

Q162
Scenario: A data page referenced in many rules needs a schema change without breaking consumers.

Question: **How do you evolve a shared data structure safely?**

> **Ideal answer focus:** Additive, backward-compatible changes; version if breaking; impact analysis (where-used); update the canonical object; test consumers; coordinate migration; avoid silent breakage.

Q163
Scenario: A connector must handle both synchronous quick lookups and asynchronous long operations.

Question: **How do you design mixed sync/async integration patterns?**

> **Ideal answer focus:** Sync via data page for fast lookups; async via queue processor for long operations with callback/polling for results; choose per use case; don't block the UI on long calls; monitor both.

Q164
Scenario: You must guarantee that reference data used in a decision is consistent for the duration of a case.

Question: **How do you snapshot reference data for decision consistency?**

> **Ideal answer focus:** Snapshot the relevant reference values onto the case (data transform) at the decision point so later source changes don't alter the recorded decision; document the snapshot; balance vs live reference.

Q165
Scenario: A partner sends duplicate messages; your case updates must not double-apply.

Question: **How do you enforce inbound idempotency?**

> **Ideal answer focus:** Dedupe by message/event ID (store processed IDs); check-before-apply; queue processor at-least-once delivery handled idempotently; reject/ignore duplicates; audit.

Pillar C — Pega Constellation Architecture

Q166
Scenario: A team migrating from Theme Cosmos keeps trying to add custom HTML sections and non-auto-generated UI in the new app, and rendering breaks.

Question: **Why does this fail in Constellation, and what is the correct authoring model?**

> **Ideal answer focus:** Constellation is configuration over customization: UI is generated from the data model via View-based authoring; no sections/harnesses/custom HTML markup; use Views + Constellation components/templates; DX API returns UI metadata, not HTML. Guardrail: DX API compliance, view-based authoring.

Q167
Scenario: Explain to a skeptical .NET lead how the browser renders a Pega screen in Constellation with 'no server-side HTML'.

Question: **Walk through the Constellation request/render lifecycle end to end.**

> **Ideal answer focus:** MVC: minimal HTML shell + Cosmos React framework from CDN; Controller (client orchestration layer) calls DX API v2; Model (Infinity server) returns data + UI metadata (View rules), not HTML; Constellation design system (View) renders React components client-side; SPA. Stateless per call.

Q168
Scenario: Performance is poor on first load: huge JS bundles are served from the app server for every user.

Question: **How does Constellation deliver static assets efficiently, and what may be misconfigured?**

> **Ideal answer focus:** Pega UI static content via global Pega CDN (framework JS); client app assets via App-Static service; verify CDN usage, caching headers, and that assets aren't served from the app node; 30% fewer requests / smaller payload benefit only with CDN.

Q169
Scenario: A front-end team wants to build a fully custom React UI but still use Pega's case processing and rules.

Question: **What are their options and the trade-offs?**

> **Ideal answer focus:** Options: Constellation SDK (React) using ConstellationJS engine; or consume Constellation DX API directly with any framework; or Web Embed / Constellation DX components. Trade-off: SDK keeps prescriptive behavior; raw DX API = max control, more work. Keep business logic on server (center-out).

Q170
Scenario: The team asks why Constellation 'releases the clipboard after each call' and how that changes design.

Question: **How does statelessness affect application design compared to traditional Pega?**

> **Ideal answer focus:** Stateless: requestor session ends / clipboard released after each DX API call; no reliance on server-held UI state between interactions; each request self-contained; enables scalability/horizontal scaling; design for stateless interactions, pass needed context each call.

Q171
Scenario: A required custom widget (e.g., a specialized map picker) doesn't exist in the Constellation component library.

Question: **How do you extend Constellation with custom UI properly?**

> **Ideal answer focus:** Build a Constellation DX Component (React) using core presentational components + DX API (see constellation-ui-gallery); package as RAP; register the component; keep it DX-compliant; avoid hacking HTML into Views. React/web expertise required.

Q172
Scenario: Business complains the app looks 'too standard' and wants heavy pixel-level customization everywhere.

Question: **How do you set expectations about Constellation's prescriptive UI?**

> **Ideal answer focus:** Constellation is prescriptive/configuration-first (40 yrs best practice, consistency, upgrade-safety); theming/design tokens allow branding; deep pixel customization fights the model and raises cost/maintenance; reserve custom DX components for true gaps.

Q173
Scenario: Explain the difference between the Constellation DX API and the Traditional (v1) DX API to a new architect.

Question: **What distinguishes v2 (Constellation) from v1, and when is each used?**

> **Ideal answer focus:** v1 exposes section/harness (UI-Kit/Theme Cosmos) processes; v2 is view-based, returns richer metadata (layouts, fields, validation, visibility, actions), supports native + custom UIs, center-out, new endpoints (follow case, change stage). v2 for Constellation apps only.

Q174
Scenario: A view shows a field conditionally, but the front end must not even receive data the user can't see.

Question: **How does Constellation handle conditional visibility and data exposure?**

> **Ideal answer focus:** Visibility/validation/actions are part of the DX API view metadata; server evaluates visibility; sensitive data gated by security (access control) not just visibility; viewType controls metadata returned. Don't rely on client-side hiding for security.

Q175
Scenario: The mobile experience must automatically adapt layout and behavior versus desktop from the same Views.

Question: **How do you drive channel-specific behavior in Constellation?**

> **Ideal answer focus:** x-origin-channel header lets rules be circumstanced by channel (web/mobile); responsive Constellation components adapt; same DX endpoints, different circumstanced Views/behavior; keep logic center-out.

Q176
Scenario: A DX API PUT to update a case intermittently fails with a 409 conflict in a multi-user scenario.

Question: **What causes this and how should the client handle it?**

> **Ideal answer focus:** Optimistic locking: eTag (pxSaveDateTime) + if-match header; a stale eTag => 409 conflict; client must refetch latest eTag/data, merge, and retry; design UX for conflict; this is intended concurrency control in Constellation.

Q177
Scenario: The front end needs to render a case creation form but is fetching far more metadata than necessary, slowing the screen.

Question: **How do you control how much UI metadata the DX API returns?**

> **Ideal answer focus:** viewType query parameter (e.g., form vs page vs none) controls metadata volume; request only what's needed; pageName to target the right top-level page (default pyDetails); tune calls to reduce payload.

Q178
Scenario: A developer is confused about which page the DX API reads/writes for a custom top-level page name.

Question: **How does the DX API know which page to operate on?**

> **Ideal answer focus:** Default top-level page is pyDetails; use the pageName query parameter when the top-level page differs; content targets fields, pageInstructions target embedded lists/groups; understand request-body structure.

Q179
Scenario: Leadership wants to embed a live Pega case widget inside their existing corporate website.

Question: **Which Constellation capability fits and what are the constraints?**

> **Ideal answer focus:** Web Embed (Constellation) to embed Pega-driven experiences; or Constellation SDK; authentication/SSO and CORS considerations; channel independence via DX API; keep business logic server-side.

Q180
Scenario: The ConstellationJS engine and SDK come up in an architecture review; the team doesn't know their role.

Question: **What is ConstellationJS and why does it matter for custom front ends?**

> **Ideal answer focus:** ConstellationJS is the engine between the DX API and the UI layer (handles orchestration/state/rendering logic); exposed via SDKs so custom front ends get Constellation behavior without hand-coding against raw DX API; ensures consistency.

Q181
Scenario: A traditional app has custom controls, auto-generated=false sections, and pyWorkPage assumptions everywhere.

Question: **What is your Constellation-readiness assessment approach?**

> **Ideal answer focus:** Run DX API readiness/compliance checks; identify non-compliant UI (custom HTML, non-auto sections, unsupported controls), harness/section dependencies, and pyWorkPage/session assumptions; plan remediation to Views + data model; screen layout as harness container.

Q182
Scenario: Separation of concerns is cited as a Constellation benefit; a stakeholder wants concrete meaning.

Question: **How does Constellation enforce separation of concerns, and why does it aid maintainability/upgrades?**

> **Ideal answer focus:** MVC split: Model (rules/data/logic on Infinity), View (design system components), Controller (client orchestration); UI metadata separated from rendering; business logic never encoded in channels; upgrades to design system don't touch logic; easier maintenance.

Q183
Scenario: An upgrade broke a heavily customized traditional UI; leadership wants to avoid this in future.

Question: **How does Constellation reduce upgrade risk compared to traditional UI?**

> **Ideal answer focus:** Prescriptive components maintained by Pega + delivered via CDN; configuration (data model) generates UI; fewer brittle customizations; custom DX components isolated and versioned; separation of concerns limits blast radius of platform updates.

Q184
Scenario: The team wants server-side validation to surface instantly in the custom React UI.

Question: **How do validation and actions flow from server to a Constellation/custom UI?**

> **Ideal answer focus:** DX API responses carry validation messages, conditional visibility, and available actions as metadata; the client renders them; server remains source of truth; a rule run (e.g., validate) returns messages the client displays; no logic duplicated client-side.

Q185
Scenario: A performance review shows many small chatty DX calls on one screen.

Question: **How do you optimize DX API interaction patterns?**

> **Ideal answer focus:** Batch/structure Views to minimize round trips; use appropriate viewType; leverage the richer single-response metadata (Constellation returns full view definition); lazy-load heavy/independent data; cache static assets via CDN.

Q186
Scenario: A client needs a design-system-consistent look but with their brand colors, fonts, and logo.

Question: **How do you brand a Constellation app without breaking prescriptiveness?**

> **Ideal answer focus:** Theming via design tokens / app-level styling; brand assets via App-Static service; stay within the design system; avoid overriding component internals; keep upgrade-safety.

Q187
Scenario: A single-page-application concern is raised: how does deep-linking/bookmarking a specific case work?

Question: **How does Constellation handle routing/navigation as an SPA?**

> **Ideal answer focus:** Client-side routing in the SPA maps URLs to Views/case context; DX API loads the corresponding case/assignment; the shell loads once then rewrites content; support deep links to case/assignment IDs.

Q188
Scenario: The org wants both an out-of-the-box Constellation portal AND a bespoke customer-facing UI from the same app.

Question: **How do you serve two experiences from one Pega application?**

> **Ideal answer focus:** Same DX API powers OOTB portal and custom front end; build custom UI via SDK/DX API while staff use the Constellation portal; shared rules/logic; channel independence; consistent behavior across both.

Q189
Scenario: A developer asks whether they can still use activities/data transforms with Constellation.

Question: **What server-side rules remain relevant, and what changes in Constellation?**

> **Ideal answer focus:** Server-side logic (data transforms, decisions, declaratives, flows, data pages, validate) still applies and is preferred; what changes is the UI layer (Views not sections/harnesses) and the stateless DX API interaction; center-out unchanged.

Q190
Scenario: Constellation is being adopted but some complex legacy screens can't be reproduced yet.

Question: **How do you phase a migration when full parity isn't immediately possible?**

> **Ideal answer focus:** Prioritize high-value journeys as Constellation Views; keep/replace legacy incrementally; where custom is needed, build DX components; avoid mixing incompatible UI models on one screen; roadmap remediation; validate DX compliance progressively.

Q191
Scenario: The team wants to reuse a set of fields/layout across many Views consistently.

Question: **How do you promote UI reuse in the view-based model?**

> **Ideal answer focus:** Reusable Views embedded across steps; field definitions on the data object drive consistent presentation; templates/patterns from the design system; relevant records; configuration-driven consistency.

Q192
Scenario: Security asks how the stateless model affects session security and CSRF/token handling.

Question: **What are the security implications of Constellation's stateless DX API?**

> **Ideal answer focus:** OAuth 2.0 bearer tokens per request; no long-lived server UI session state; standard API security (token expiry/refresh, CORS, TLS); access control enforced server-side each call; design for token lifecycle.

Q193
Scenario: A stakeholder claims Constellation 'removes the need for architects'.

Question: **How do you articulate the architect's role in a prescriptive, generated-UI world?**

> **Ideal answer focus:** Architect focuses on data model, case design, integration, security, performance, DX compliance, and when/where custom DX components are justified; prescriptive UI shifts effort from pixel-building to solid modelling and center-out design.

Q194
Scenario: Rendering differs subtly between the OOTB portal and a custom SDK front end.

Question: **Why might behavior differ, and how do you keep parity?**

> **Ideal answer focus:** If the custom front end consumes raw DX API vs using ConstellationJS/SDK, orchestration/behavior may differ; prefer the SDK/engine for parity; ensure same DX endpoints/metadata; test both against the same Views.

Q195
Scenario: A view must show a computed/derived value that updates as the user edits other fields.

Question: **How does client-side reactivity work with server-authoritative logic in Constellation?**

> **Ideal answer focus:** Declarative logic (Declare Expression) computed server-side; the client may re-fetch/refresh view metadata on change (Controller orchestrates DX calls); actions/refresh triggered by field events; server stays authoritative. Avoid client-side business logic.

Q196
Scenario: A custom Constellation DX component must call a Pega data view and render results in a bespoke layout.

Question: **How do you build a data-driven custom component correctly?**

> **Ideal answer focus:** Constellation DX Component (React) using the DX API data view endpoints and core presentational components; follow the constellation-ui-gallery patterns; package/register as a component; stay DX-compliant; keep logic server-side.

Q197
Scenario: The team wants to know how much UI metadata vs data comes back from a single DX API call for a view.

Question: **How is the DX API response structured, and how does that aid custom rendering?**

> **Ideal answer focus:** Single response carries data + full UI metadata (layouts, fields, validation, conditional visibility, actions) so any framework can reproduce the View; richer than v1; reduces round trips; viewType tunes metadata depth.

Q198
Scenario: A stakeholder asks why Constellation reduced network traffic so dramatically vs the old UI.

Question: **What architectural choices drive Constellation's network efficiency?**

> **Ideal answer focus:** CDN-served static framework assets; SPA (load shell once, rewrite content); metadata-rich responses reducing round trips (~30% fewer first-load requests, ~7x smaller payload); stateless calls; client-side rendering.

Q199
Scenario: A React front end must drive a case from creation through resolution using only APIs.

Question: **Which DX API endpoints and sequence accomplish full case processing?**

> **Ideal answer focus:** Case type/create endpoints (with caseTypeID, content, pageInstructions), assignment/next-assignment endpoints, view endpoints, submit/perform actions, and case actions (change stage, follow); sequence them per the workflow; handle eTag between updates.

Q200
Scenario: A view must insert several rows into an embedded page list in one save from a custom UI.

Question: **How does the DX API handle nested list mutations?**

> **Ideal answer focus:** pageInstructions array in the request body describing insert/update/delete operations on the embedded page list/group; content handles scalar fields; nested data cannot be set by plain assignment — pageInstructions are required.

Q201
Scenario: Leadership fears vendor lock-in to Pega's UI and wants portability of the front end.

Question: **How does Constellation's architecture address front-end flexibility?**

> **Ideal answer focus:** DX API is open REST + framework-agnostic; Constellation SDK/ConstellationJS for React; you can extend or replace the front end while keeping server-side logic; center-out keeps logic out of channels; portability by design.

Q202
Scenario: An architecture board asks how client-side orchestration keeps business logic authoritative.

Question: **How does Constellation prevent business logic leaking into the client?**

> **Ideal answer focus:** Controller only orchestrates DX calls and rendering; all rules/decisions/validation/visibility computed server-side and returned as metadata; the client never owns business logic; center-out principle; enforce on server each stateless call.

Q203
Scenario: A screen intermittently shows stale action buttons after a server-side stage change.

Question: **How should a Constellation/custom client stay in sync with server state?**

> **Ideal answer focus:** Re-fetch view/case metadata after actions (the DX response reflects new available actions/status); rely on server-returned metadata, not cached client assumptions; use eTag to detect changes; refresh on action completion.

Q204
Scenario: The team wants to unit/regression test the front end independently of the server.

Question: **How do you test a Constellation custom front end effectively?**

> **Ideal answer focus:** Mock DX API responses (metadata + data) to test rendering; Storybook/Playwright for components (as in the gallery); contract tests against DX schema; separate UI tests from server rule tests (PegaUnit).

Q205
Scenario: A performance trace shows the browser re-rendering excessively on each keystroke.

Question: **How do you optimize client-side rendering behavior in a custom Constellation UI?**

> **Ideal answer focus:** Debounce inputs, minimize unnecessary DX calls, use appropriate refresh scope (refresh only affected view parts), leverage the design system's optimized components; avoid full-view refresh per change.

Q206
Scenario: A client insists on server-side rendering for SEO on a public-facing Pega experience.

Question: **How do you reconcile Constellation's client-rendered SPA with SSR/SEO needs?**

> **Ideal answer focus:** Constellation is a client-rendered SPA; for SEO/public content consider a separate presentation layer consuming DX API with SSR, or Web Embed within an SSR site; discuss trade-offs; keep Pega as the logic/data backend.

Q207
Scenario: Two Views need to share complex conditional visibility logic that must stay consistent.

Question: **How do you keep visibility/validation logic DRY across Views?**

> **Ideal answer focus:** Define the condition once (when rule) and reference it in each View's visibility; server evaluates and returns it in metadata; avoid duplicating logic in client code; configuration-driven.

Q208
Scenario: An upgrade to the Constellation design system subtly changed a component's spacing, affecting layouts.

Question: **How do you manage design-system version changes safely?**

> **Ideal answer focus:** Design system delivered/versioned by Pega via CDN; test against new versions in lower envs; rely on tokens/theming rather than hardcoded spacing; isolate custom components; regression-test UI; follow release notes.

Q209
Scenario: A field must render as a specialized control not offered out of the box, on many Views.

Question: **How do you introduce a reusable custom control across the app?**

> **Ideal answer focus:** Build a reusable Constellation DX component, register it, and use it wherever the field appears; package via RAP; keep it DX-compliant and themable; avoid per-View hacks; document usage.

Q210
Scenario: The security team asks how the client authenticates to the DX API on every stateless call.

Question: **How does authentication work for Constellation/DX API interactions?**

> **Ideal answer focus:** OAuth 2.0 bearer token per request; token acquired via the auth service (SSO/OIDC); refresh handling; no server UI session; TLS; CORS config for cross-origin custom front ends; authorization enforced server-side each call.

Q211
Scenario: A custom front end must show a case's available next actions dynamically as the workflow changes.

Question: **How does a client discover valid actions at runtime?**

> **Ideal answer focus:** The DX API returns available assignment/case actions in the view/case metadata; the client renders actions from that metadata rather than hardcoding; re-fetch after each action; server governs what's allowed.

Q212
Scenario: A team wants to progressively adopt Constellation without rewriting every legacy screen at once.

Question: **What are viable coexistence strategies during migration?**

> **Ideal answer focus:** Prioritize Constellation Views for key journeys; keep legacy where needed but avoid mixing incompatible UI on one screen; use DX components for gaps; roadmap remediation to DX compliance; validate readiness incrementally.

Q213
Scenario: A stakeholder asks how Constellation achieves consistency across dozens of teams building apps.

Question: **How does the prescribed design system enforce consistency?**

> **Ideal answer focus:** Shared library of prescribed components/patterns and templates; configuration over customization generates consistent UI from the data model; theming for brand only; reduces divergent custom builds.

Q214
Scenario: A view must conditionally require a field only when another field has a certain value, enforced everywhere.

Question: **How is conditional validation expressed and delivered to clients?**

> **Ideal answer focus:** Validation/required conditions defined server-side (validate/when), returned as view metadata via DX API; the client renders and enforces display, server enforces on submit; consistent across OOTB and custom UIs.

Q215
Scenario: A performance audit finds the app loads many components it never displays.

Question: **How do you reduce unnecessary component/metadata loading?**

> **Ideal answer focus:** Structure Views to load only what's shown; lazy-load heavy/independent views; appropriate viewType; avoid over-broad views; leverage SPA on-demand rendering; measure payloads.

Q216
Scenario: A custom component needs to trigger a case action (e.g., approve) via the API.

Question: **How does a component invoke server-side workflow actions?**

> **Ideal answer focus:** Call the DX API assignment/case action endpoints (submit flow action / case action) with content/pageInstructions and eTag; the server processes the flow and returns updated metadata; render the result; handle conflicts.

Q217
Scenario: The org wants analytics on how users interact with the Constellation UI.

Question: **How do you capture UX telemetry in a Constellation app?**

> **Ideal answer focus:** Client-side instrumentation in the front end / SDK; integrate with analytics tooling; capture events without leaking PII; correlate with case data server-side; respect privacy.

Q218
Scenario: A custom front end and the OOTB portal must show identical validation and behavior.

Question: **How do you guarantee behavioral parity between clients?**

> **Ideal answer focus:** Both consume the same DX API metadata (validation/visibility/actions computed server-side); prefer ConstellationJS/SDK for orchestration parity; single source of logic; test both against the same Views.

Q219
Scenario: A Constellation app must support accessibility (WCAG) requirements.

Question: **How does Constellation help meet accessibility, and what must you still do?**

> **Ideal answer focus:** Prescribed components are built for accessibility; still test with tooling (as the gallery does accessibility testing); ensure custom DX components meet WCAG; proper labels/semantics; don't defeat built-in accessibility with hacks.

Q220
Scenario: Leadership questions why not just build a bespoke React app hitting Pega APIs and skip Constellation entirely.

Question: **What do you gain from Constellation vs a fully bespoke front end on the DX API?**

> **Ideal answer focus:** Constellation gives prescribed, upgrade-safe, consistent UI + orchestration (ConstellationJS) out of the box, faster delivery, less maintenance; bespoke = full control but high build/maintenance cost and lost prescriptiveness; DX API underpins both.

Q221
Scenario: A view's data comes from a data page that is slow; the rest of the screen should render immediately.

Question: **How do you avoid one slow data source blocking view render?**

> **Ideal answer focus:** Structure the View so the slow data loads in a separate/deferred view or on demand; the DX API returns per-view data; progressive rendering; lazy-load; set timeouts; isolate the slow dependency.

Q222
Scenario: A custom component must respect the same theming/branding as OOTB components.

Question: **How do custom components inherit the design system's look?**

> **Ideal answer focus:** Build on core Constellation presentational components and design tokens so theming applies automatically; avoid hardcoded styles; package via the standard component model; stays consistent through theme/upgrade changes.

Q223
Scenario: A public API consumer sends malformed requests to the DX API.

Question: **How does the DX API and your design guard against bad client requests?**

> **Ideal answer focus:** Server validates request structure (content/pageInstructions), method, headers (if-match), and authorization; returns appropriate errors; the client must handle error responses; never trust client input; enforce server-side.

Q224
Scenario: A team wants to know how state is maintained if the server releases the clipboard after each call.

Question: **How is continuity maintained across stateless DX interactions?**

> **Ideal answer focus:** Each request carries needed context (case ID, page, content, eTag); server rebuilds context per call and persists case state in the DB; no reliance on in-memory session; the client holds transient UI state; design accordingly.

Q225
Scenario: A custom front end must handle optimistic-lock conflicts gracefully when two users edit a case.

Question: **How should the client implement the eTag/if-match conflict flow?**

> **Ideal answer focus:** Send if-match with the last eTag; on 409, fetch the latest data/eTag, show the user the conflict/merge, then retry; never silently overwrite; this is Constellation's concurrency model.

Q226
Scenario: A view should render differently for an internal agent vs an external customer from the same case.

Question: **How do you deliver audience-specific views over the DX API?**

> **Ideal answer focus:** Persona/role-driven Views; circumstance by channel (x-origin-channel) where needed; DX API returns the appropriate view metadata per context; security filters data; shared logic, tailored presentation.

Q227
Scenario: A team wants to reuse Pega's orchestration but embed only a single case widget in a portal.

Question: **Which Constellation option fits embedding a slice of functionality?**

> **Ideal answer focus:** Web Embed (Constellation) or the SDK to embed a specific case/view; DX API powers it; handle auth/SSO and CORS; keep it consistent with OOTB behavior; channel independence.

Q228
Scenario: A custom component must localize its labels to the user's language.

Question: **How do you localize custom Constellation components?**

> **Ideal answer focus:** Use client-side localization assets (App-Static service) aligned with Pega localization; pull locale from context; keep text externalized; test locales; consistent with platform localization.

Q229
Scenario: An architecture review asks how Constellation supports horizontal scalability.

Question: **Why does the stateless, client-orchestrated model scale well?**

> **Ideal answer focus:** Stateless DX calls release the clipboard/session, so any node can serve any request; no sticky session state; CDN offloads static assets; client does rendering; enables elastic horizontal scaling.

Q230
Scenario: A custom UI must display validation errors inline exactly as Pega computed them.

Question: **How do server-computed messages reach and render in the client?**

> **Ideal answer focus:** The DX API response includes validation messages tied to fields; the client maps and renders them inline; server is authoritative; no client-side re-implementation of business validation.

Q231
Scenario: A component must conditionally enable an action based on server-side rules, not client guesses.

Question: **How do you drive action enablement from the server?**

> **Ideal answer focus:** Available actions and their conditions come in the DX metadata; the client enables/disables from that; re-fetch after state changes; never hardcode enablement logic client-side.

Q232
Scenario: The team debates storing UI state on the server for a wizard across calls.

Question: **Given statelessness, where does multi-step state live?**

> **Ideal answer focus:** Case data persists server-side in the DB; per-step content is sent with each DX call (content/pageInstructions + eTag); transient UI state stays on the client; no server session reliance between calls.

Q233
Scenario: A Constellation app must integrate a third-party charting library in a custom component.

Question: **How do you safely integrate external libraries into DX components?**

> **Ideal answer focus:** Bundle within the DX component (React), vet/pin/scan the dependency, keep it isolated and versioned, follow the gallery build practices; ensure accessibility/theming; limit to genuine need.

Q234
Scenario: A stakeholder wants proof the front end won't break when Pega updates the design system.

Question: **How does Constellation limit UI upgrade breakage?**

> **Ideal answer focus:** Prescribed components maintained by Pega via CDN; configuration-driven UI regenerates; theming via tokens; custom components isolated/versioned and tested against new releases; separation of concerns limits blast radius.

Q235
Scenario: A view must load a large related dataset only when the user expands a section.

Question: **How do you defer heavy view data until needed?**

> **Ideal answer focus:** On-demand/lazy view loading triggered by the expand action (a DX call for that view); the initial response stays light; progressive disclosure; keeps first render fast.

Q236
Scenario: A custom front end must support deep-linking directly into a specific assignment.

Question: **How do you route directly to an assignment in a Constellation SPA?**

> **Ideal answer focus:** Client routing maps the URL to the assignment ID; DX API loads that assignment's view/context; the SPA renders it; handle auth and not-found; support bookmarking.

Pillar D — Security, Automation & Performance

Q237
Scenario: A nightly agent-based job from a legacy app is unreliable after upgrade and doesn't scale across nodes.

Question: **How do you modernize background processing in current Pega?**

> **Ideal answer focus:** Replace agents with Queue Processors (async, dedicated/standard) for event/message-driven work and Job Schedulers for time-based recurring work; they're node-cluster aware, scalable, and monitored in Admin Studio; use Kafka-backed queues. Guardrail: use QP/Job Scheduler, not agents.

Q238
Scenario: A queue processor is falling behind; messages pile up and SLAs on downstream cases are breached.

Question: **How do you diagnose and tune a lagging queue processor?**

> **Ideal answer focus:** Monitor via Admin Studio (queue size, throughput, failures); tune thread count/concurrency, node allocation, and message size; check for slow per-item processing (PAL); handle poison messages/DLQ; scale nodes; ensure idempotency.

Q239
Scenario: A background process intermittently reprocesses the same item, causing duplicate side effects.

Question: **How do you guarantee exactly-once-effect semantics in async processing?**

> **Ideal answer focus:** Design idempotency (dedupe keys, check-before-act); queue processors provide at-least-once delivery, so handle retries safely; use transactional markers; avoid non-idempotent external calls without guards.

Q240
Scenario: Under load, a specific interaction takes 8 seconds; the team blames the database but isn't sure.

Question: **How do you methodically find the bottleneck?**

> **Ideal answer focus:** PAL (Performance Analyzer) for elapsed vs CPU vs DB vs connector time per interaction; DB Trace for slow SQL; Tracer for rule-level cost; PDC for cluster trends; isolate the dominant contributor before optimizing. Measure, don't guess.

Q241
Scenario: Reports and Get-Next-Work slowed dramatically as data volume grew.

Question: **What are the likely causes and fixes for query performance at scale?**

> **Ideal answer focus:** Un-optimized (BLOB) properties used in filters/joins => expose/optimize to columns + indexes; DB Trace to find slow SQL; report on exposed/declare-indexed data; partitioning/archival; avoid unindexed filtering.

Q242
Scenario: A production incident shows one interaction consuming huge memory and threatening node stability.

Question: **How do you find and remediate memory/requestor issues?**

> **Ideal answer focus:** PAL/PDC and memory diagnostics; look for oversized clipboard pages, huge data pages (wrong scope), unbounded lists; right-size data page scope; page cleanup; avoid loading large datasets into memory.

Q243
Scenario: A rule change 'works for me' but a subset of users get the old behavior.

Question: **How does rule resolution explain per-user differences, and how do you debug it?**

> **Ideal answer focus:** Rule resolution: ruleset stack (per access group/app), class hierarchy, availability, circumstance, date, security; different users => different stack/circumstance/data; use Tracer + rule 'Resolution' info; check circumstanced variants and ruleset versions.

Q244
Scenario: Two rulesets contain a rule with the same name and the wrong one is winning at runtime.

Question: **How do you control which rule resolves?**

> **Ideal answer focus:** Ruleset stack order (application ruleset list), version, availability (Available/Blocked/Withdrawn), circumstance specificity, and class specificity; adjust stack/version or withdraw; understand resolution precedence; avoid duplicate ambiguity.

Q245
Scenario: An SLA escalation activity is running but escalation emails aren't sent under load.

Question: **How do you troubleshoot SLA/escalation processing?**

> **Ideal answer focus:** SLAs are processed by the standard SLA queue processor (pzInternalSLA...); check that QP is running and not backlogged (Admin Studio); verify escalation activity, correspondence, and calendar; Tracer the SLA agent/QP; check node allocation.

Q246
Scenario: Sensitive fields must be invisible to some roles and encrypted at rest, with row-level restrictions by region.

Question: **How do you layer Pega security controls to meet this?**

> **Ideal answer focus:** RBAC (access groups/roles/AROs) for class/action access; ABAC (access control policies) for row/field-level dynamic restrictions by attribute (region); property encryption / platform cipher for at-rest; UI visibility is not security. Defense in depth.

Q247
Scenario: A privilege-gated action is still executable via the DX API by crafting a request.

Question: **How do you ensure security isn't bypassed at the API layer?**

> **Ideal answer focus:** Enforce authorization server-side (privileges, AROs, access control policies) on the rule/case, not just UI visibility; DX API respects server security; never rely on hiding the control; validate on every request (stateless).

Q248
Scenario: The company mandates SSO with SAML and fine-grained authorization from an external policy system.

Question: **How do you architect authentication and authorization integration?**

> **Ideal answer focus:** Authentication service (SAML 2.0 / OIDC) for SSO; map external attributes to Pega roles/access groups; ABAC policies for fine-grained rules; token-based (OAuth) for API; keep policy externalized where required; least privilege.

Q249
Scenario: An operator's access group grants far more than their job needs, a compliance finding.

Question: **How do you implement least-privilege access cleanly?**

> **Ideal answer focus:** Persona-based access (Access Manager / persona access landing page); roles scoped to job function; AROs granting minimal levels; Access Deny for explicit blocks; periodic review; avoid over-broad access groups.

Q250
Scenario: A batch of cases must be processed at midnight, but only once, even across a multi-node cluster.

Question: **How do you ensure single-execution scheduled work across nodes?**

> **Ideal answer focus:** Job Scheduler configured for the cluster (runs on one node per schedule) rather than per-node agents; verify node classification/scheduling; idempotent logic as a safety net; monitor execution.

Q251
Scenario: A long-running synchronous operation in a flow blocks the user and risks DX API timeouts.

Question: **How do you offload long work without blocking the interaction?**

> **Ideal answer focus:** Move to async: queue processor / background; return control to the user; notify on completion; the case resumes via Wait/callback; keep DX API interactions short (stateless).

Q252
Scenario: Tracer is too noisy to find the issue in a complex production-like scenario.

Question: **How do you use Tracer effectively for targeted diagnosis?**

> **Ideal answer focus:** Filter by ruleset/rule type/event, watch specific pages/rules, limit to the relevant requestor/case, enable declarative/DB where needed; combine with PAL for cost; use remote tracing/PDC in shared environments; avoid tracing everything.

Q253
Scenario: Password/lockout and session policies must meet a strict security standard.

Question: **Where and how do you enforce platform-level security policies?**

> **Ideal answer focus:** Security policies (authentication lockout, password complexity, CAPTCHA, session settings); operator authentication config; audit logging; align with standards; use platform settings rather than custom.

Q254
Scenario: An external penetration test flags potential injection and access issues in custom rules.

Question: **How do you harden a Pega app against common vulnerabilities?**

> **Ideal answer focus:** Follow Pega security guardrails: parameterize/validate inputs, avoid custom SQL/HTML/Java, use access control policies, enable output escaping, keep platform patched, review the security checklist; run the security scan.

Q255
Scenario: A queue processor item fails repeatedly and blocks the queue.

Question: **How do you handle poison messages and failure isolation?**

> **Ideal answer focus:** Configure retry limits and broken-item/dead-letter handling; move failed items to a failure queue for inspection; alert; ensure one bad item doesn't stall throughput; monitor failures in Admin Studio.

Q256
Scenario: Performance is fine in load tests but degrades over hours in production.

Question: **How do you investigate gradual degradation?**

> **Ideal answer focus:** PDC trends, memory/GC patterns, growing data page caches, un-expired caches, DB growth without archival, connection pool exhaustion; look for leaks/unbounded growth; schedule archival; right-size caches.

Q257
Scenario: The team wants automated regression to prevent rule changes from breaking case behavior.

Question: **How do you build quality gates into the delivery pipeline?**

> **Ideal answer focus:** PegaUnit test cases + test coverage; run in Deployment Manager CI/CD pipeline; guardrail compliance thresholds; block promotion on failures; scenario tests for flows; shift-left quality.

Q258
Scenario: A dynamic system setting change didn't take effect and caused confusion about caching.

Question: **How do configuration changes propagate, and how do you avoid stale config?**

> **Ideal answer focus:** Understand DSS/ setting caching and when a cache refresh/requestor reset is needed; some settings require reload; use application settings; test propagation; document.

Q259
Scenario: An access control policy for row-level security is slowing down large report queries.

Question: **How do you balance ABAC security with query performance?**

> **Ideal answer focus:** ABAC policy conditions add query predicates; ensure the driving properties are optimized/indexed; scope policies narrowly; test with DB Trace; balance security granularity vs cost; consider policy design.

Q260
Scenario: A scheduled job overlaps with the previous run because it takes longer than its interval.

Question: **How do you prevent overlapping executions of recurring work?**

> **Ideal answer focus:** Job Scheduler configuration to avoid overlap / run-on-one-node; make runs idempotent; monitor duration vs interval; chunk work; alert on long runs.

Q261
Scenario: A high-volume case type generates millions of rows; users report timeouts opening worklists.

Question: **How do you keep operational queries fast at very high volume?**

> **Ideal answer focus:** Expose/optimize + index key columns; archive resolved cases; use Get-Next-Work efficiently; report via Insights on optimized/indexed data; DB partitioning; avoid full-table scans; monitor with DB Trace/PDC.

Q262
Scenario: Encryption is required but developers worry about performance and search.

Question: **How do you apply encryption without crippling functionality?**

> **Ideal answer focus:** Property-level encryption / platform cipher for sensitive fields; understand you can't filter/sort on encrypted columns easily; encrypt only what's needed; keep searchable fields as controlled non-sensitive; balance compliance vs function.

Q263
Scenario: A custom Java step was added for 'speed' but tanks the guardrail score and risks upgrades.

Question: **How do you justify removing custom code, and what replaces it?**

> **Ideal answer focus:** Guardrails: custom Java reduces compliance, complicates upgrades, and isn't optimized by the engine; replace with configured rules (data transforms, decisions, declaratives, functions); reserve Java for genuinely unsupported needs; raise compliance score.

Q264
Scenario: An SLA-driven urgency scheme isn't reflecting business priority; VIP cases don't surface first.

Question: **How do you engineer prioritization to reflect true business value?**

> **Ideal answer focus:** Design urgency via SLA initial urgency + escalations + manual adjustments; incorporate priority attributes in Get-Next-Work; possibly decisioning for prioritization; measure and tune; keep configurable.

Q265
Scenario: A production issue must be diagnosed but Tracer/PAL on prod is risky.

Question: **How do you diagnose safely in production?**

> **Ideal answer focus:** Use PDC (Predictive Diagnostic Cloud) and logs for non-invasive insight; remote tracing scoped to one requestor; reproduce in a prod-like lower env; avoid broad tracing under load; use alerts/AES/PDC dashboards.

Q266
Scenario: A stakeholder wants proof that the app will scale to 10x users before a marketing launch.

Question: **How do you approach performance/scalability validation as an architect?**

> **Ideal answer focus:** Load/performance testing to targets; analyze with PAL/PDC; identify bottlenecks (DB, connectors, memory, data pages); tune scope/indexes/async; validate CDN/stateless scaling in Constellation; capacity planning and headroom.

Q267
Scenario: A queue processor must process items in strict order for a given account but in parallel across accounts.

Question: **How do you achieve per-key ordering with overall parallelism?**

> **Ideal answer focus:** Partition by key (account) so ordering holds within a partition while different partitions process concurrently; design the queue/partition strategy; avoid global serialization; idempotency; monitor throughput.

Q268
Scenario: An SLA-driven escalation storm fires thousands of emails at once after a backlog clears.

Question: **How do you prevent escalation floods and smooth processing?**

> **Ideal answer focus:** Throttle/queue escalation actions; batch notifications; ensure the SLA queue processor isn't reprocessing en masse; design digest notifications; monitor; avoid synchronous mass sends.

Q269
Scenario: A report used by executives times out during business hours but is fine at night.

Question: **How do you make heavy reporting coexist with transactional load?**

> **Ideal answer focus:** Optimize/index queried columns; schedule/materialize heavy reports off-peak; read replicas/reporting nodes where available; Insights on optimized data; caching; avoid unindexed filters; DB Trace to tune SQL.

Q270
Scenario: Rule resolution is picking a circumstanced rule when it shouldn't for certain users.

Question: **How do you debug unexpected circumstance resolution?**

> **Ideal answer focus:** Inspect circumstance definitions (property/date/template) and their inputs per user; check circumstance specificity precedence; Tracer resolution details; verify data driving the circumstance; consolidate overlapping circumstances.

Q271
Scenario: A withdrawn rule in a higher ruleset is unexpectedly still affecting behavior.

Question: **How do withdraw/availability states interact with the ruleset stack?**

> **Ideal answer focus:** Withdraw removes a rule at a stack level so resolution falls through to a lower one; verify which ruleset the withdraw lives in and stack order; Blocked vs Withdrawn vs Available; check resolution path with Tracer.

Q272
Scenario: A background job holds a case lock too long, blocking users from working the case.

Question: **How do you avoid lock contention between background and interactive work?**

> **Ideal answer focus:** Minimize lock scope/duration; use optimistic locking where suitable; process without holding locks unnecessarily; release promptly; design async work to avoid long-held case locks; monitor lock waits.

Q273
Scenario: PAL shows high 'Rules Executed' count and elapsed time on a seemingly simple screen.

Question: **How do you interpret PAL readings to target optimization?**

> **Ideal answer focus:** Break down elapsed into DB, connector, rule execution, and declarative processing; high rule counts suggest excessive declarative recalculation or repeated data page loads; target the dominant cost; compare readings before/after changes.

Q274
Scenario: A data page with node scope is being reloaded far more often than expected, spiking source load.

Question: **How do you diagnose unexpected data page reloads?**

> **Ideal answer focus:** Check reload-on-condition logic, scope misconfiguration (thread/requestor vs node), cache expiry, and whether parameters vary (creating many instances); Tracer data page load events; fix scope/keys/reload strategy.

Q275
Scenario: Security requires that all case data access be attributable and auditable, including API access.

Question: **How do you ensure comprehensive audit across UI and API?**

> **Ideal answer focus:** Case history (pxHistory), security audit logging, field-level auditing where needed; DX API access tied to authenticated operator; log security events; avoid gaps between channels; retention policy.

Q276
Scenario: A third-party library was added to a custom DX component; security flags supply-chain risk.

Question: **How do you govern custom front-end dependencies?**

> **Ideal answer focus:** Vet/pin dependencies, scan for vulnerabilities, review licenses, keep components isolated and versioned, follow the gallery's build/test practices; limit custom components to genuine needs; governance/review.

Q277
Scenario: An access group change accidentally exposed an admin portal to standard users.

Question: **How do you prevent and detect over-privileged access changes?**

> **Ideal answer focus:** Least-privilege personas/roles; change control on access groups/roles; periodic access reviews; test access as each persona; Access Deny for sensitive functions; audit configuration changes.

Q278
Scenario: A job scheduler task must only run in production, not in lower environments.

Question: **How do you control where scheduled/background work executes?**

> **Ideal answer focus:** Node classification / environment-aware settings gating execution; DSS/application settings to enable per environment; verify node types; avoid running prod jobs in test; document.

Q279
Scenario: A spike in DX API traffic from a custom front end degrades the whole cluster.

Question: **How do you protect the platform from client-driven load?**

> **Ideal answer focus:** Rate limiting/throttling on API; efficient client call patterns (batching, viewType); autoscaling stateless nodes; monitor with PDC; CDN for static assets; enforce good DX interaction design.

Q280
Scenario: A validate rule runs expensive logic on every keystroke via client refresh.

Question: **How do you avoid over-invoking server logic from the UI?**

> **Ideal answer focus:** Trigger validation on blur/submit rather than each keystroke; scope refresh to needed fields; move heavy checks to submit; debounce; keep per-interaction cost low given stateless calls.

Q281
Scenario: Leadership wants a proactive view of production health rather than reacting to incidents.

Question: **How do you set up proactive performance and health monitoring?**

> **Ideal answer focus:** PDC (Predictive Diagnostic Cloud) for alerts/trends; dashboards for queue processors/job schedulers (Admin Studio); SLA/backlog monitoring; capacity trends; alerting thresholds; regular guardrail/compliance review.

Q282
Scenario: A rule that resolves correctly in dev resolves to a different version in production for the same user.

Question: **How do you reconcile environment-specific rule resolution differences?**

> **Ideal answer focus:** Compare ruleset versions/stack promoted per environment, availability states, and locked vs unlocked rulesets; verify the same rules were migrated (RAP/Deployment Manager); check circumstance data; use resolution info in each env.

Q283
Scenario: A queue processor's throughput must scale for a seasonal spike then scale back down.

Question: **How do you scale asynchronous processing elastically?**

> **Ideal answer focus:** Add nodes / adjust queue processor thread allocation; autoscaling stateless nodes; monitor backlog in Admin Studio; tune concurrency; ensure idempotency so scaling doesn't cause duplicates; scale down after.

Q284
Scenario: An access control policy must restrict access based on a relationship (agent owns the customer).

Question: **How do you model relationship-based row-level security?**

> **Ideal answer focus:** ABAC access control policy comparing operator context to case/customer attributes (ownership); ensure driving properties are optimized/indexed for performance; test; combine with RBAC; server-enforced.

Q285
Scenario: A performance regression appeared after adding several Declare Expressions.

Question: **How do you assess and control declarative processing cost?**

> **Ideal answer focus:** Forward chaining recalculates on input change; excessive/interdependent expressions add cost; use PAL to measure declarative time; switch rarely-read values to backward chaining; simplify the dependency network.

Q286
Scenario: Sensitive audit logs must be retained for 7 years but kept out of hot operational tables.

Question: **How do you manage long-term retention without hurting performance?**

> **Ideal answer focus:** Archival/purge policies moving old data to archive storage; separate reporting/archive from operational tables; retention configuration; ensure audit integrity; partition; monitor DB growth.

Q287
Scenario: A background job must process only cases meeting complex criteria efficiently at scale.

Question: **How do you drive high-volume selection efficiently?**

> **Ideal answer focus:** Query on optimized/indexed columns (not BLOB); paginate/stream; queue processor for parallel processing; avoid loading all into memory; DB Trace to validate SQL; idempotent per-item handling.

Q288
Scenario: A security review requires that no business logic is executable without an authenticated, authorized context.

Question: **How do you guarantee authorization on every execution path including APIs and background work?**

> **Ideal answer focus:** RBAC/ABAC enforced at the rule/case level; service packages authenticated (OAuth); queue/job context runs under a defined access group with least privilege; no anonymous privileged execution; audit.

Q289
Scenario: A production node shows thread exhaustion during peak; users get errors.

Question: **How do you diagnose and relieve thread/connection exhaustion?**

> **Ideal answer focus:** PDC/JMX diagnostics; identify long-running requests, slow connectors, and connection pool sizing; async offload of long work (queue processor); tune pools; autoscale; fix the slow root cause, not just capacity.

Q290
Scenario: A rule change must be promoted with zero risk of breaking dependent rules.

Question: **How do you manage rule dependencies and safe promotion?**

> **Ideal answer focus:** Impact analysis (where-am-I-used / dependency tooling); PegaUnit regression + coverage; branch + merge; Deployment Manager pipeline with gates; ruleset versioning; guardrail check; roll back plan.

Q291
Scenario: An SLA must consider priority differently for VIP vs standard customers, tuned by business.

Question: **How do you make SLA/prioritization business-configurable?**

> **Ideal answer focus:** Drive SLA intervals/urgency from a decision table + application settings keyed on customer tier; circumstance the SLA if needed; keep it editable by business; measure and iterate; avoid hard-coding.

Q292
Scenario: A custom activity performs a database operation that bypasses Pega's persistence, causing inconsistencies.

Question: **Why is this dangerous and what is the correct approach?**

> **Ideal answer focus:** Bypassing the engine breaks caching/locking/declarative integrity and guardrails; use Obj-* methods / data pages / savable data pages / proper persistence; avoid raw SQL/custom DB writes; keep the engine authoritative.

Q293
Scenario: You need to confirm a change actually improved performance, not just felt faster.

Question: **How do you validate optimization objectively?**

> **Ideal answer focus:** Baseline with PAL/PDC before, apply change, re-measure the same scenario; compare DB/connector/rule/elapsed breakdown; controlled load test; document deltas; avoid anecdotal 'seems faster'.

Q294
Scenario: A job scheduler and queue processor both touch the same records and occasionally conflict.

Question: **How do you coordinate concurrent async workloads on shared data?**

> **Ideal answer focus:** Locking/ownership strategy; partition responsibilities so they don't overlap; idempotency; sequence via status/flags; monitor for contention; avoid two processors racing on the same records.

Q295
Scenario: A penetration test found that error messages leak internal details to end users.

Question: **How do you handle errors securely and gracefully?**

> **Ideal answer focus:** Catch and translate errors to user-safe messages; never expose stack traces/internal IDs; log details server-side (without PII); consistent error handling in flows/integrations/DX responses; align with security guardrails.

Q296
Scenario: Leadership wants assurance the architecture is upgrade-ready and low-maintenance long term.

Question: **How do you keep an application maintainable and upgrade-safe as an architect?**

> **Ideal answer focus:** High guardrail compliance (minimize custom Java/HTML/activities); configuration over customization; DX-API-compliant view-based UI; reuse via ECS/relevant records; automated tests; isolated versioned custom components; regular health/compliance review.

Q297
Scenario: A queue processor must guarantee that a delayed message is processed at a future time.

Question: **How do you schedule delayed asynchronous work?**

> **Ideal answer focus:** Delayed queue processing (queue an item with a future run time) or a job scheduler; avoid busy-waiting; idempotency; monitor; distinguish event-driven (QP) vs time-based (Job Scheduler).

Q298
Scenario: Rule resolution must prefer a client's implementation rule over a framework rule reliably.

Question: **How does the ruleset stack guarantee override precedence?**

> **Ideal answer focus:** The implementation ruleset sits above the framework in the application's ruleset list, so resolution finds it first; verify stack order, versions, and availability; test resolution; avoid ambiguous duplicates.

Q299
Scenario: A performance issue is intermittent and only reproduces under concurrency.

Question: **How do you diagnose concurrency-related performance/locking issues?**

> **Ideal answer focus:** PDC/PAL under load; look for lock contention, DB deadlocks (DB Trace), connection pool limits, and shared-resource hotspots; reproduce with load tests; reduce lock scope/async offload.

Q300
Scenario: An SLA queue processor backlog is silently growing and only noticed when escalations stop.

Question: **How do you proactively monitor system-managed queue processors?**

> **Ideal answer focus:** Admin Studio dashboards + PDC alerts on queue depth/failures for standard QPs (including SLA); set thresholds; node allocation; investigate slow items; capacity planning.

Q301
Scenario: A sensitive operation must be permitted only from specific network origins.

Question: **How do you enforce context-based access beyond user role?**

> **Ideal answer focus:** Combine authentication/authorization with network/origin controls (channel via x-origin-channel, IP allowlisting at the edge, auth service conditions); ABAC on context attributes; defense in depth; server-enforced.

Q302
Scenario: A memory leak is suspected because heap grows until restart resolves it.

Question: **How do you investigate and address gradual memory growth?**

> **Ideal answer focus:** PDC/heap analysis; look for growing data page caches (wrong scope/no expiry), unbounded clipboard pages, retained references; right-size scopes/expiry; fix the growth source; schedule archival; avoid restart-as-fix.

Q303
Scenario: A rule must be effective only during a promotional window and revert automatically after.

Question: **How do you time-bound rule behavior declaratively?**

> **Ideal answer focus:** Date-based circumstancing/availability so the variant is effective only in the window and the base applies otherwise; no manual toggling; test boundaries; audit.

Q304
Scenario: A background job must not run if the previous run failed and left data inconsistent.

Question: **How do you guard scheduled work against unsafe re-runs?**

> **Ideal answer focus:** State/flag guarding start; detect prior-failure state and halt/alert; idempotent recovery; don't blindly re-run; monitor; design for safe restart.

Q305
Scenario: An audit requires proving who could access a given case class and why.

Question: **How do you demonstrate and reason about effective access?**

> **Ideal answer focus:** Trace access via roles/AROs/access control policies for the class; use access reporting/tooling; document RBAC + ABAC layers; periodic review; least privilege; auditable configuration.

Q306
Scenario: A connector's retries under an outage amplify load and worsen the outage.

Question: **How do you prevent retry storms?**

> **Ideal answer focus:** Bounded retries with exponential backoff + jitter; circuit breaker to stop calling a failing endpoint; queue and defer; alert; avoid tight synchronous retry loops; monitor.

Q307
Scenario: A report's ABAC filtering plus large volume makes it unusably slow.

Question: **How do you optimize a secured high-volume query?**

> **Ideal answer focus:** Ensure policy-driving and filter columns are optimized/indexed; narrow policy scope; pre-aggregate/materialize; report off-peak or on a reporting node; DB Trace to tune; balance security granularity vs cost.

Q308
Scenario: Leadership wants confidence that async processing won't lose messages during a deployment.

Question: **How do you ensure durability across deployments?**

> **Ideal answer focus:** Durable (Kafka-backed) queue processors persist messages; graceful shutdown/drain during deploy; at-least-once delivery + idempotency; monitor for stuck/failed items; test deploy scenarios; DLQ for failures.